INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040
SYSTEM PROGRAMMING

WEEK 14 LECTURE 1

CONCURRENT PROGRAMMING

Concurrent Programming
BASICS

Concurrent Programming is Hard!

- The human mind tends to be sequential
- The notion of time is often misleading
- Thinking about all possible sequences of events in a computer system is at least error prone and frequently impossible

Concurrent Programming

- Logical control flows are concurrent if they overlap in time.
- This general phenomenon, known as concurrency, shows up at many different levels of a computer system.
- Hardware exception handlers, processes, and Unix signal handlers are all familiar examples.

Concurrent Programming

- Concurrency is considered mainly as a mechanism that the operating system kernel uses to run multiple application programs.
- But concurrency is not just limited to the kernel.
- It can play an important role in application programs as well.
- For example, Unix signal handlers allow applications to respond to asynchronous events such as the user typing ctrl-c or the program accessing an undefined area of virtual memory.
- Application-level concurrency is useful in other ways as well:
  - Accessing slow I/O devices
  - Interacting with humans
  - Reducing latency by deferring work
  - Servicing multiple network clients
  - Computing in parallel on multi-core machines

Concurrent Programming

- Applications that use application-level concurrency are known as **concurrent programs.**
- Modern operating systems provide three basic approaches for building concurrent programs:
  - Processes
  - I/O Multiplexing
  - Threads

Concurrent Programming

- **Processes**

- **With this approach, each logical control flow is a process that is scheduled and maintained by the kernel.**
- **Since processes have separate virtual address spaces, flows that want to communicate with each other must use some kind of explicit inter-process communication (IPC) mechanism.**

Concurrent Programming

- **I/O multiplexing**
  - This is a form of concurrent programming where applications explicitly schedule their own logical flows in the context of a single process.
  - Logical flows are modelled as state machines that the main program explicitly transitions from state to state as a result of data arriving on file descriptors.
  - Since the program is a single process, all flows share the same address space.

Example : Event-driven Network Client-server Concurrent Pogramming

Concurrent Programming

- **Threads**

- **Threads are logical flows that run in the context of a single process and are scheduled by the kernel.**

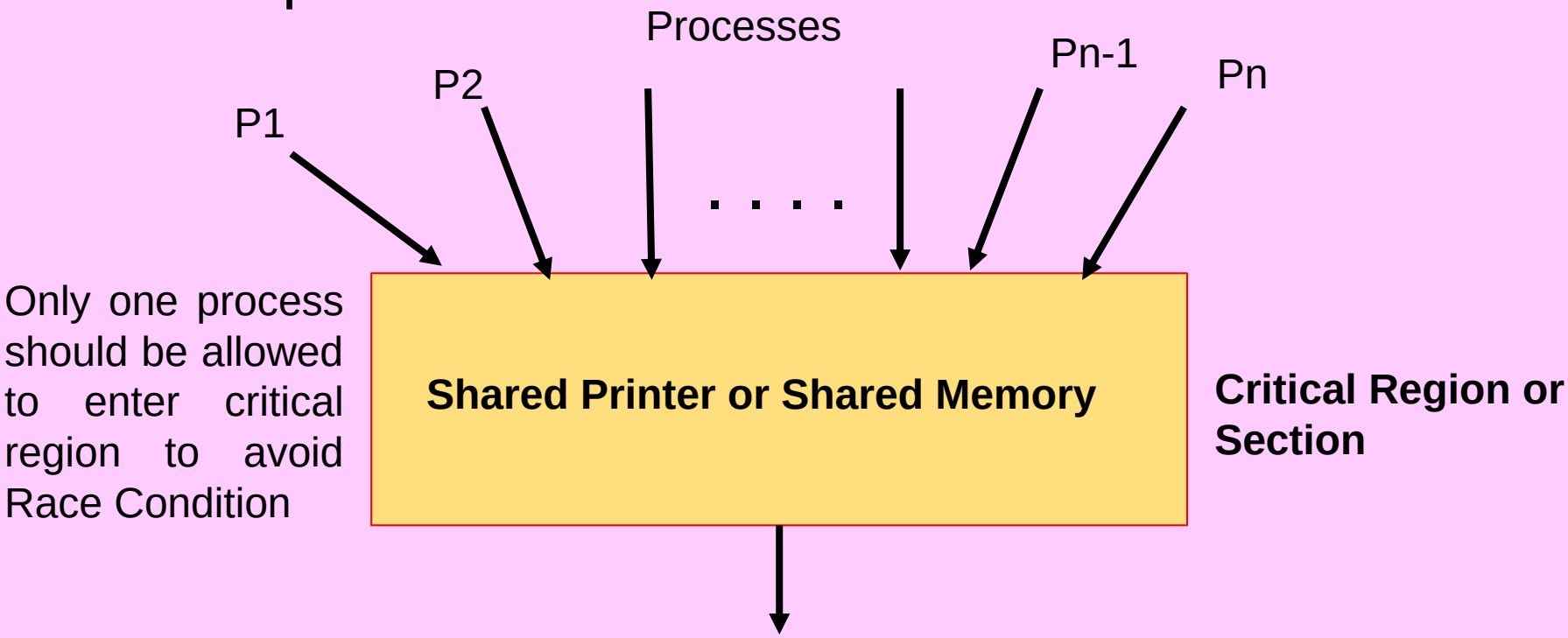
- **You can think of threads as a hybrid of the other two approaches, scheduled by the kernel like process flows, and sharing the same virtual address space like I/O multiplexing flows.**

Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Races***
  - ***Deadlock***
  - ***Livelock / Starvation / Fairness***

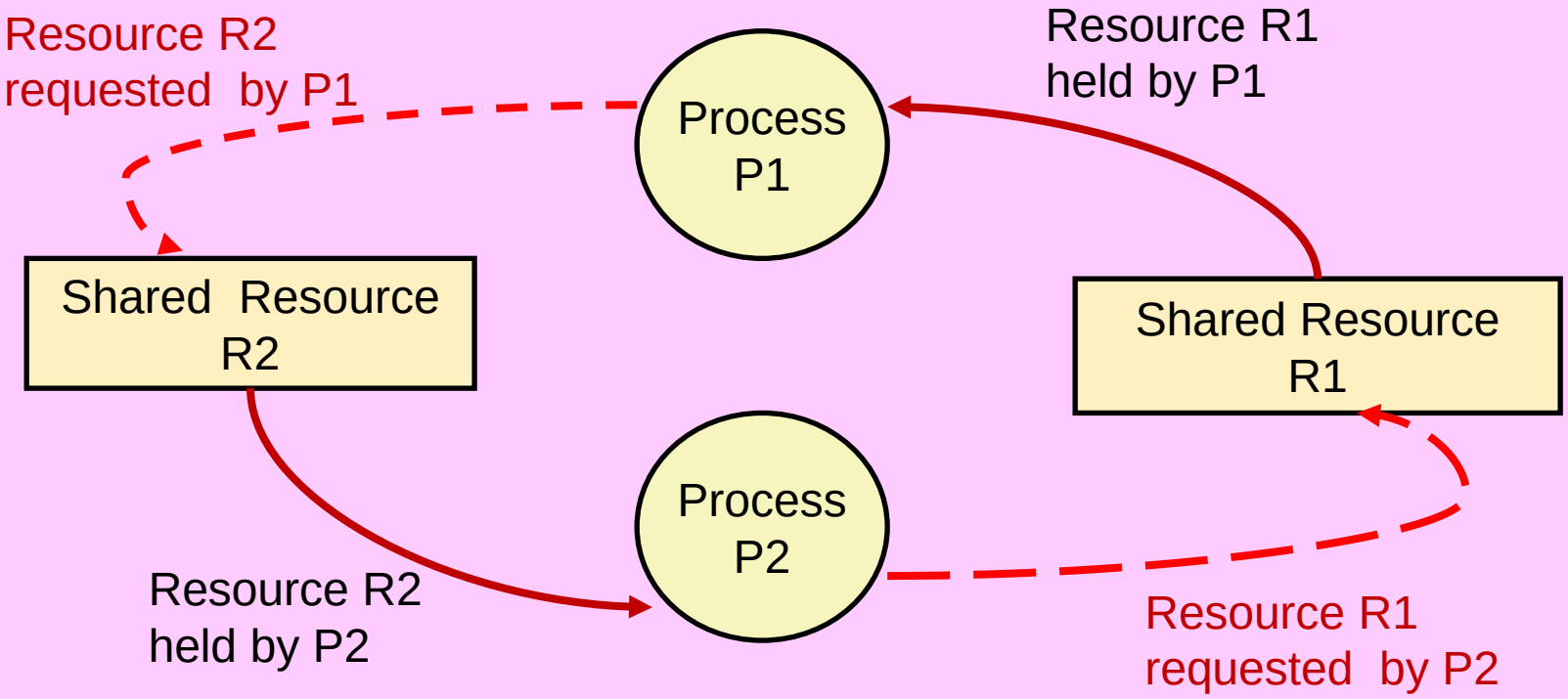
Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Races:*** outcome depends on arbitrary scheduling decisions elsewhere in the system
    - Example: who gets the last seat on the airplane?



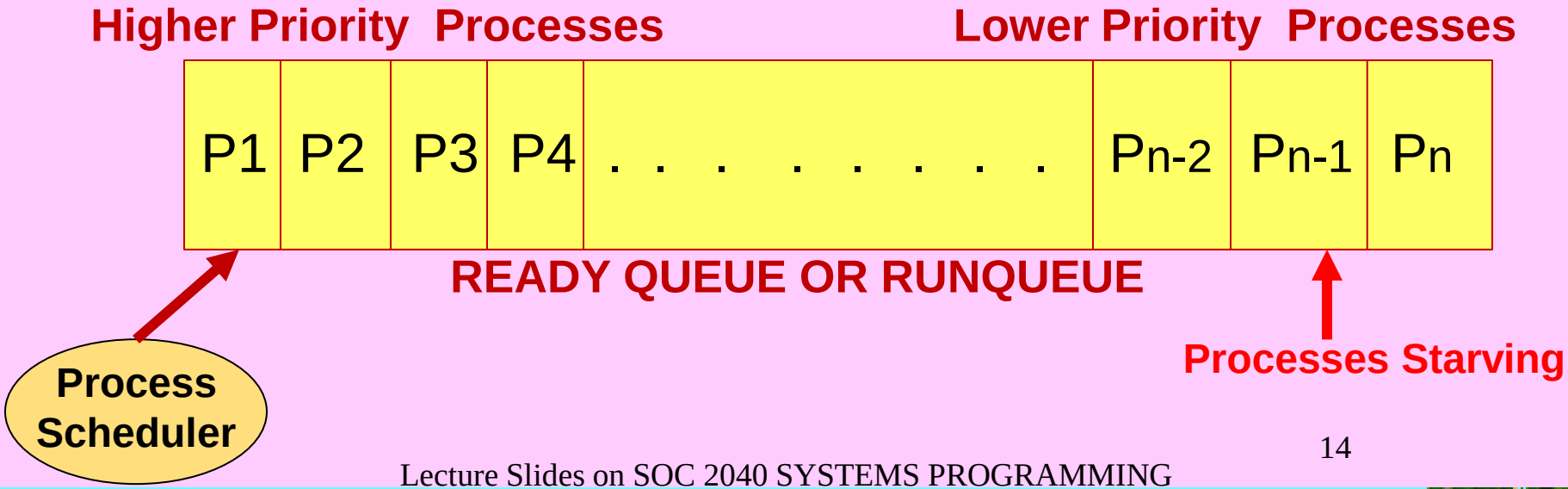
Concurrent Programming is Hard!

- Classical problem classes of concurrent programs:
  - ***Deadlock:*** improper resource allocation prevents forward progress
    - Example: traffic gridlock



Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Livelock / Starvation / Fairness***: external events and/or system scheduling decisions can prevent sub-task progress
    - Example: people always jump in front of you in line or queue



Traditional View of a Process

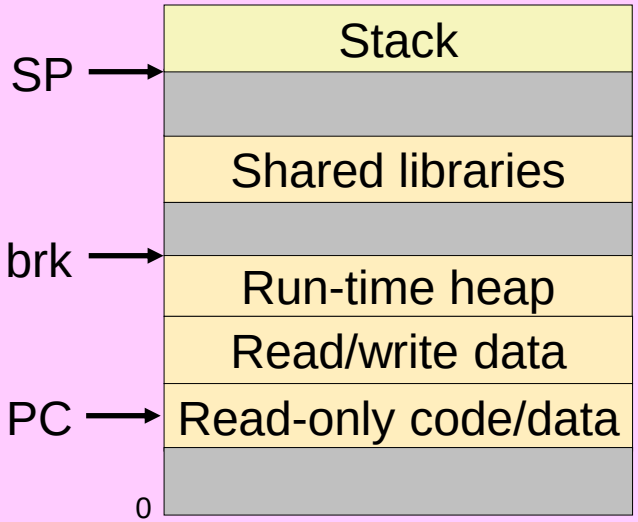
- Process = process context + code, data, and stack

Process context

Program context:
- Data registers
- Condition codes
- Stack pointer (SP)
- Program counter (PC)

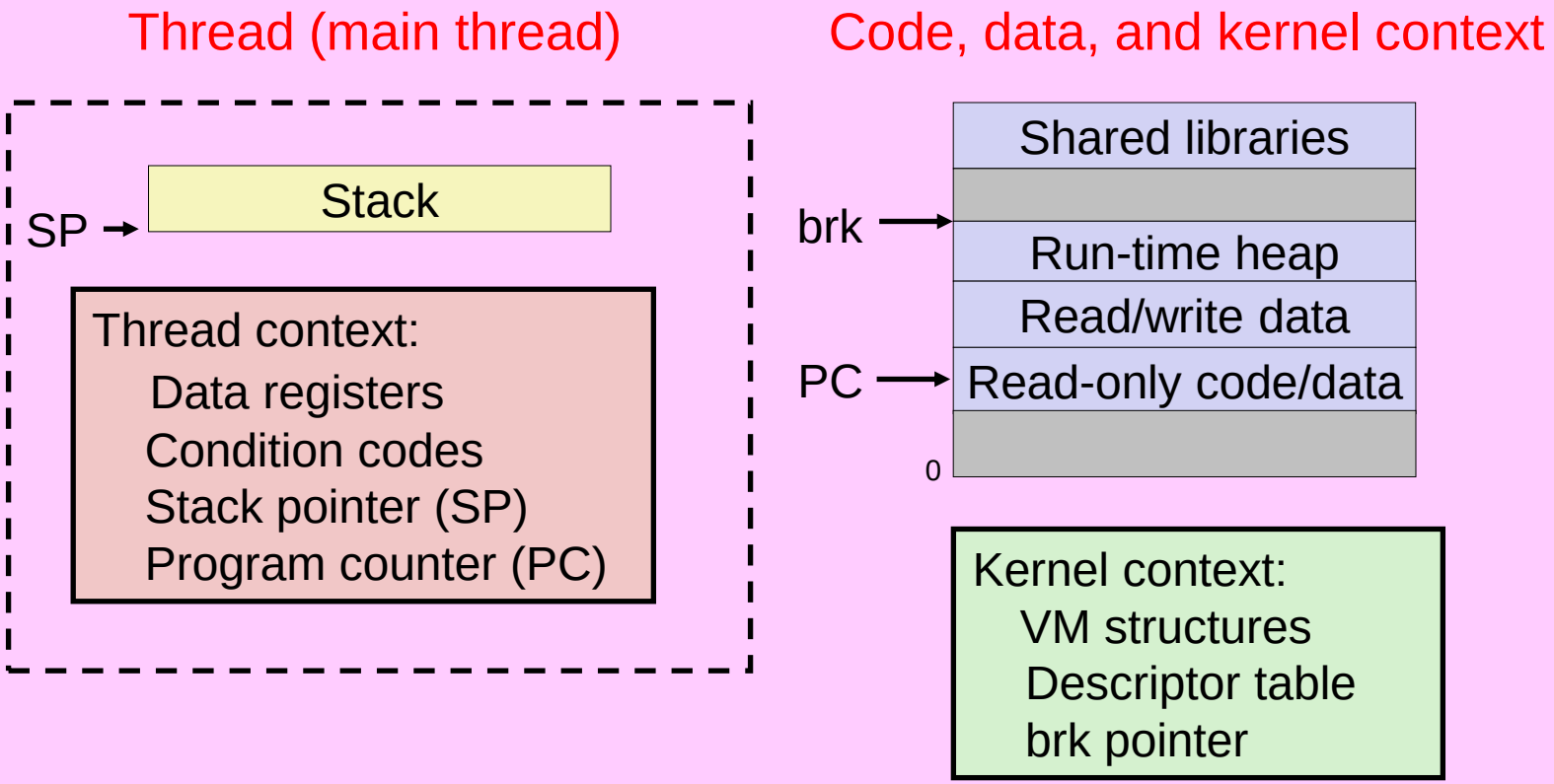
Kernel context:
- VM structures
- Descriptor table
- brk pointer

Code, data, and stack



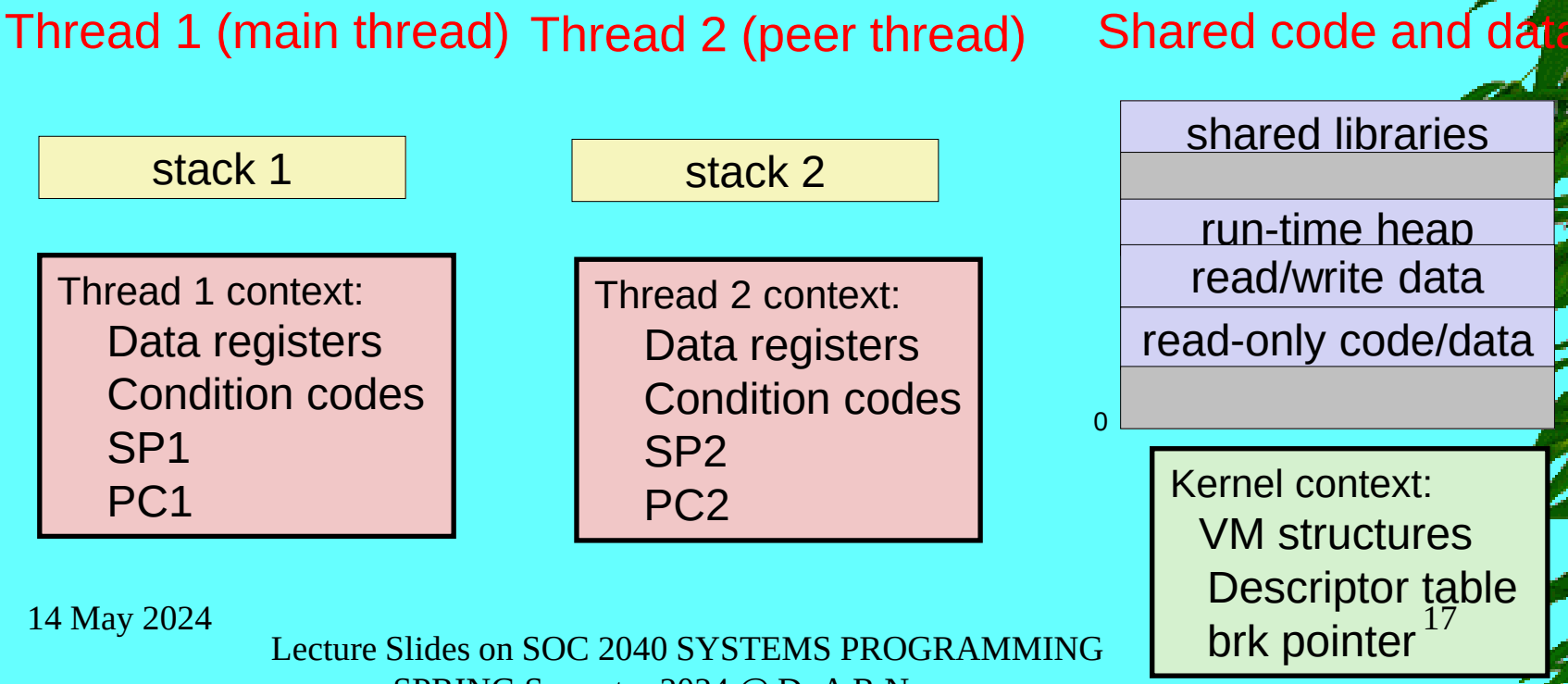
Alternate View of a Process

- Process = thread + code, data, and kernel context



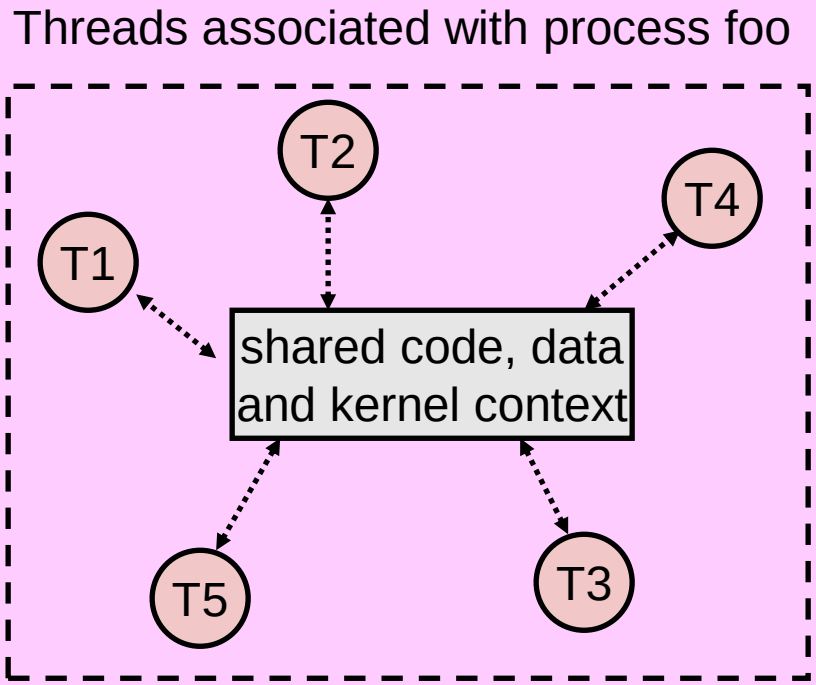
A Process With Multiple Threads

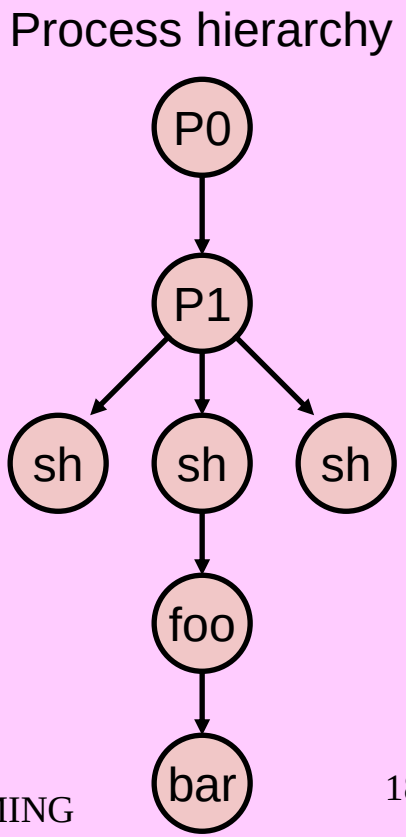
- Multiple threads can be associated with a process
  - Each thread has its own logical control flow
  - Each thread shares the same code, data, and kernel context
  - Each thread has its own stack for local variables
    - but not protected from other threads
  - Each thread has its own thread id (TID)



Logical View of Threads

- Threads associated with process form a pool of peers
  - Unlike processes which form a tree hierarchy

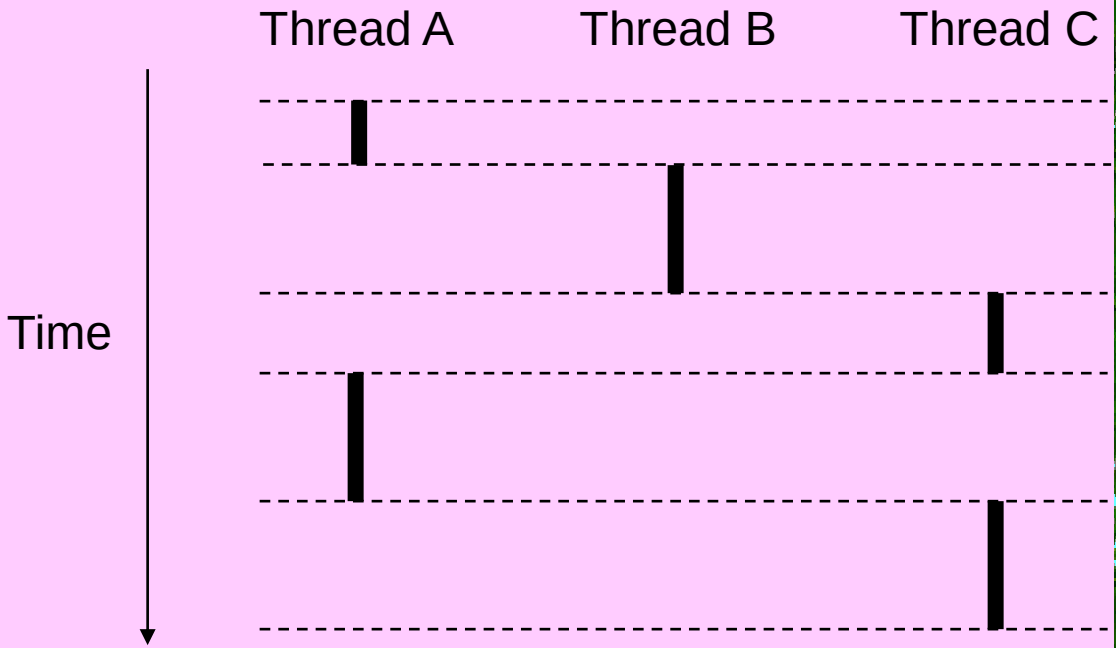




Concurrent Threads

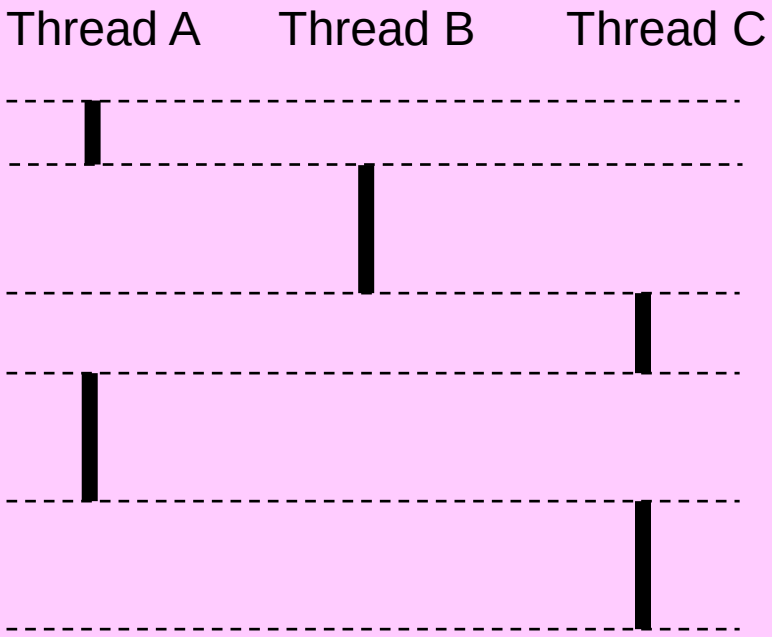
- Two threads are *concurrent* if their flows overlap in time
- Otherwise, they are sequential

- Examples:
  - Concurrent: A & B, A&C
  - Sequential: B & C

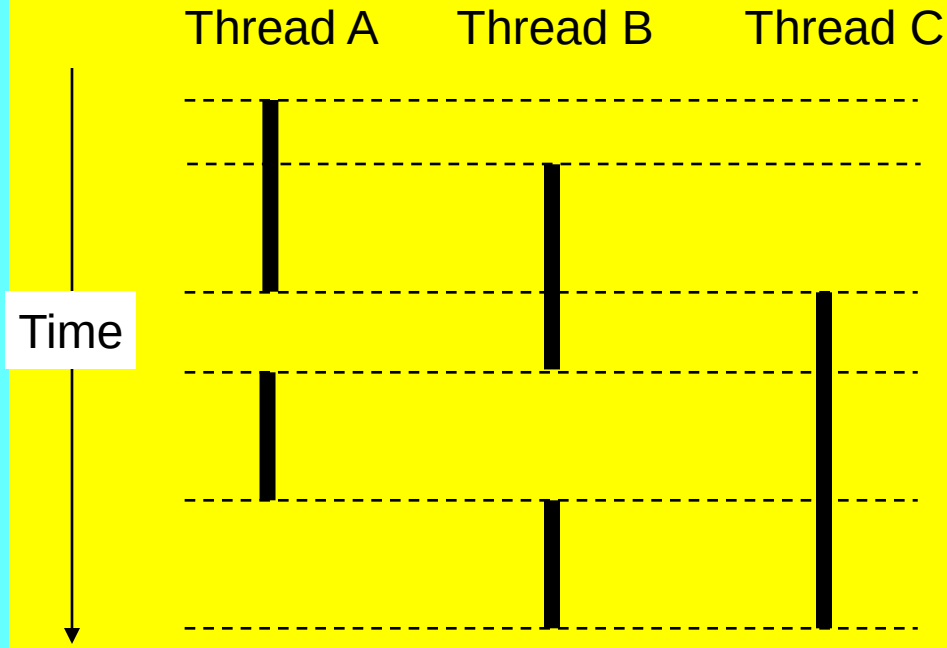


Concurrent Thread Execution

- Single Core Processor
  - Simulate parallelism by time slicing



- Multi-Core Processor
  - Can have true parallelism



Run 3 threads on 2 cores

Threads vs. Processes

- How threads and processes are similar
  - Each has its own logical control flow
  - Each can run concurrently with others (possibly on different cores)
  - Each is context switched
- How threads and processes are different
  - Threads share all code and data (except local stacks)
    - Processes (typically) do not
  - Threads are somewhat less expensive than processes
    - Process control (creating and reaping) twice as expensive as thread control
    - Linux numbers:
      - ~20K cycles to create and reap a process
      - ~10K cycles (or less) to create and reap a thread

Concurrent Programming with Threads

- A thread is a logical flow that runs in the context of a process.
- Normally programs consist of a single thread per process.
- But modern systems also allow us to write programs that have multiple threads running concurrently in a single process.
- The threads are scheduled automatically by the kernel.
- Each thread has its own thread context, including an unique integer thread ID (TID), stack, stack pointer, program counter, general-purpose registers, and condition codes.
- All threads running in a process share the entire virtual address space of that process.

Concurrent Programming with Threads

- Logical flows based on threads combine qualities of flows based on processes and I/O multiplexing.
- Like processes, threads are scheduled automatically by the kernel and are known to the kernel by an integer ID.
- Like flows based on I/O multiplexing, multiple threads run in the context of a single process, and thus share the entire contents of the process virtual address space, including its code, data, heap, shared libraries, and open files

Posix Threads (Pthreads) Interface

- *Pthreads:* Standard interface for ~60 functions that manipulate threads from C programs
  - Creating and reaping threads
    - `pthread_create()`
    - `pthread_join()`
  - Determining your thread ID
    - `pthread_self()`
  - Terminating threads
    - `pthread_cancel()`
    - `pthread_exit()`
    - `exit()` [terminates all threads] , `RET` [terminates current thread]
  - Synchronizing access to shared variables
    - `pthread_mutex_init`
    - `pthread_mutex_[un]lock`

Thread Execution Model

Creating Threads

- Threads create other threads by calling the pthread_create function.

```
#include <pthread.h>
typedef void *(func)(void *);

int pthread_create(pthread_t  *tid,  pthread_attr_t  *attr,  func *f, void *arg);
                                                    Returns: 0 if OK, nonzero on error
```

- The pthread_create function creates a new thread and runs the thread routine f in the context of the new thread and with an input argument of arg.
- The attr argument can be used to change the default attributes of the newly created thread.
- Normally pthread_create is called with a NULL attr argument.
- When pthread_create returns, argument tid contains the ID of the newly created thread.
- The new thread can determine its own thread ID by calling the pthread_self function.

```
#include <pthread.h>
pthread_t   pthread_self(void);                    Returns: thread ID of caller
```

Thread Execution Model

Terminating Threads

- A thread terminates in one of the following ways:
  - The thread terminates implicitly when its top-level thread routine returns.
  - The thread terminates explicitly by calling the pthread_exit function. If the main thread calls pthread_exit, it waits for all other peer threads to terminate, and then terminates the main thread and the entire process with a return value of thread_return.

```
#include <pthread.h>
void pthread_exit(void *thread_return);
```

- Some peer thread calls the Unix exit function, which terminates the process and all threads associated with the process.
- Another peer thread terminates the current thread by calling the pthread_ cancel function with the ID of the current thread.

```
#include <pthread.h>
int pthread_cancel(pthread_t  tid)
```

Returns: 0 if OK, nonzero on error

Thread Execution Model

Reaping Terminated Threads

- Threads wait for other threads to terminate by calling the pthread_join function.

```
#include <pthread.h>
int  pthread_join(pthread_t tid, void **thread_return)
                                        Returns: 0 if OK, nonzero on error
```

- The pthread_join function blocks until thread tid terminates, assigns the generic(void *) pointer returned by the thread routine to the location pointed to by thread_return, and then reaps any memory resources held by the terminated thread.
- Notice that, unlike the Unix wait function, the pthread_join function can only wait for a specific thread to terminate.
- There is no way to instruct pthread_ wait to wait for an arbitrary thread to terminate.
- This can complicate our code by forcing us to use other, less intuitive mechanisms to detect process termination.

Thread Execution Model

Detaching Threads

- At any point in time, a thread is joinable or detached.
- A joinable thread can be reaped and killed by other threads.
- Its memory resources (such as the stack) are not freed until it is reaped by another thread.
- In contrast, a detached thread cannot be reaped or killed by other threads.
- Its memory resources are freed automatically by the system when it terminates.
- By default, threads are created joinable.
- In order to avoid memory leaks, each joinable thread should either be explicitly reaped by another thread, or detached by a call to the pthread_detach function.

```
#include <pthread.h>
int pthread_detach(pthread_t  tid);
                                        Returns: 0 if OK, nonzero on error
```

- The pthread_detach function detaches the joinable thread tid.
- Threads can detach themselves by calling pthread_detach with an argument of pthread_ self().

Thread Execution Model

Initializing Threads

- The pthread_once function allows you to initialize the state associated with a thread routine.

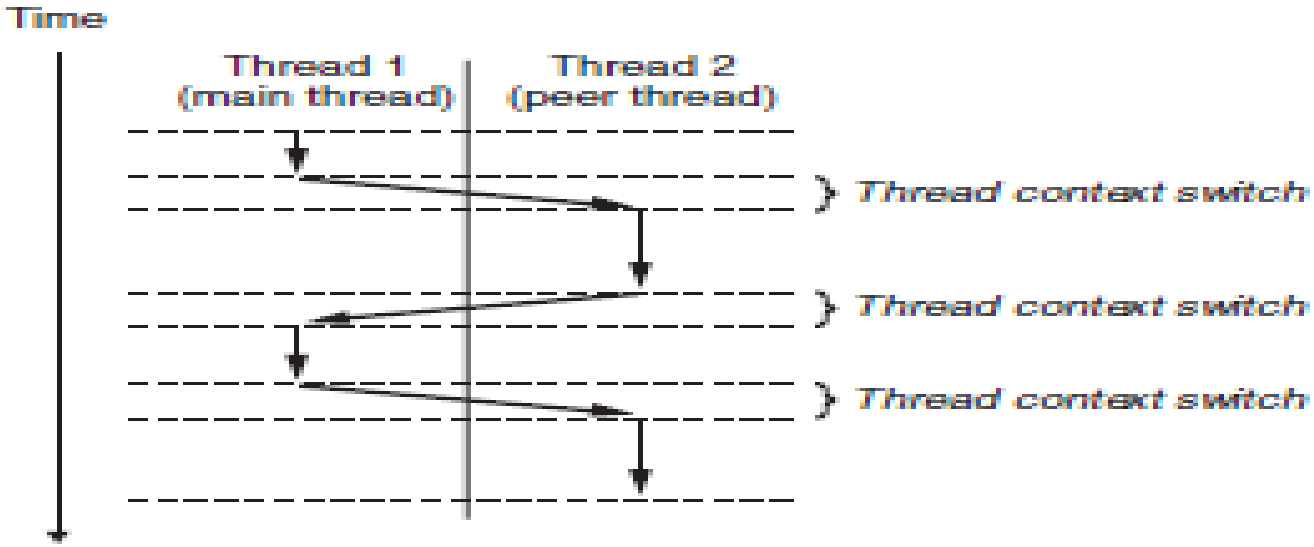
```
#include <pthread.h>
pthread_once_t  once_control = PTHREAD_ONCE_INIT;
int pthread_once(pthread_once_t  *once_control, void (*init_routine)(void));
                                                    Always returns 0
```

- The once_control variable is a global or static variable that is always initialized to PTHREAD_ONCE_INIT.
- The first time you call pthread_once with an argument of once_control, it invokes init_routine, which is a function with no input arguments that returns nothing.
- Subsequent calls to pthread_once with the same once_control variable do nothing.
- The pthread_once function is useful whenever you need to dynamically initialize global variables that are shared by multiple threads.

Thread Execution Model

- The execution model for multiple threads is similar in some ways to the execution model for multiple processes.
- Each process begins life as a single thread called the main thread.
- At some point, the main thread creates a peer thread, and from this point in time the two threads run concurrently.
- Eventually, control passes to the peer thread via a context switch, because the main thread executes a slow system call such as read or sleep, or because it is interrupted by the system's interval timer.

- The peer thread executes for a while before control passes back to the main thread, and so on.
-



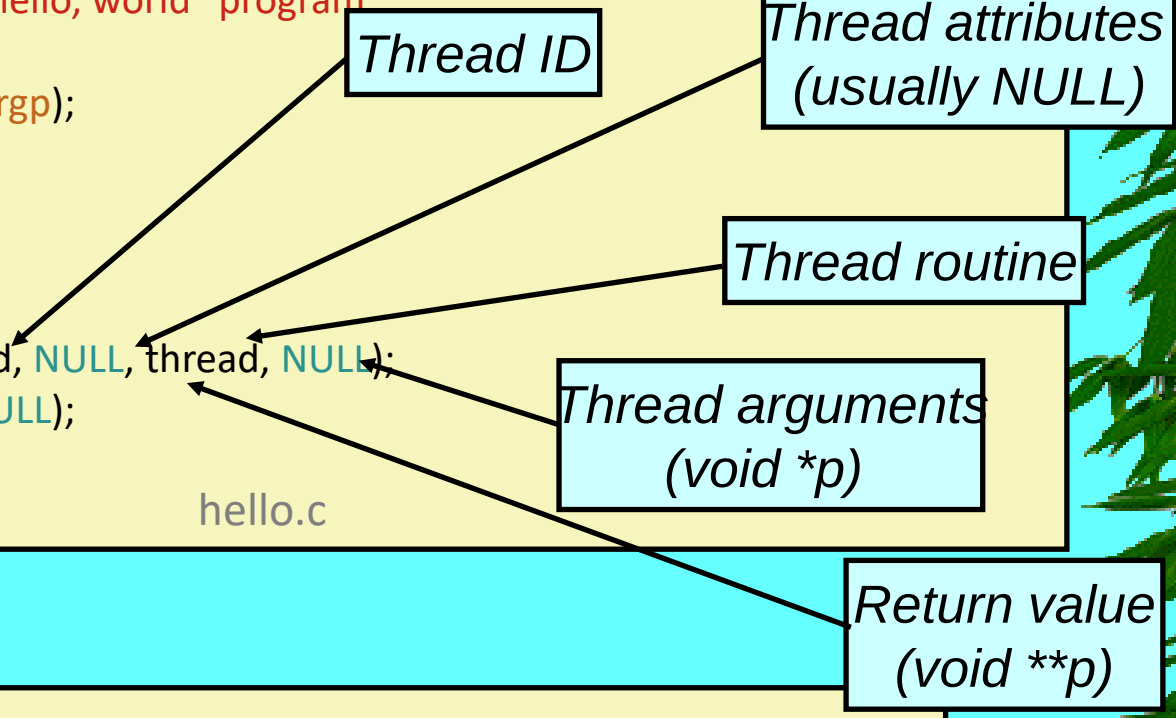
Thread Execution Model

- Thread execution differs from processes in some important ways.
- Because a thread context is much smaller than a process context, a thread context switch is faster than a process context switch.
- Another difference is that threads, unlike processes, are not organized in a rigid parent-child hierarchy.
- The threads associated with a process form a pool of peers, independent of which threads were created by which other threads.
- The main thread is distinguished from other threads only in the sense that it is always the first thread to run in the process.
- The main impact of this notion of a pool of peers is that a thread can kill any of its peers, or wait for any of its peers to terminate. Further, each peer can read and write the same shared data.

The Pthreads "hello, world" Program

```c
/*
 * hello.c - Pthreads "hello, world" program
 */
void *thread(void *vargp);

int main()
{
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    exit(0);
}
```

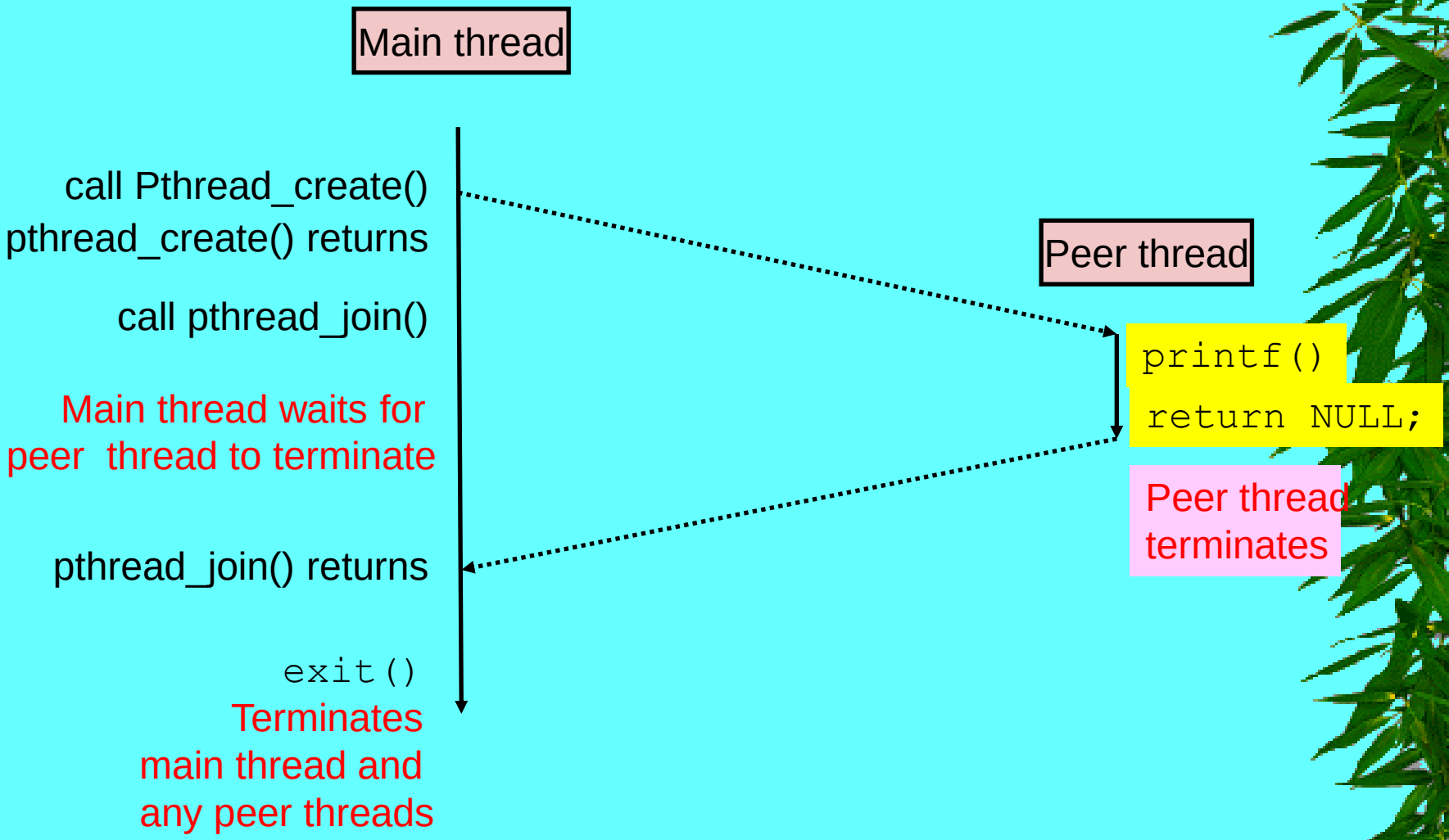
hello.c




```c
void *thread(void *vargp) /* thread routine */
{
    printf("Hello, world!\n");
    return NULL;
}
```

hello.c

Execution of Threaded “hello, world”



Thread Execution Model

- Figure shows a simple Pthreads program.
- The main thread creates a peer thread and then waits for it to terminate.
- The peer thread prints “Hello, world!\n” and terminates.
- When the main thread detects that the peer thread has terminated, it terminates the process by calling exit.
- The code and local data for a thread is encapsulated in a thread routine.
- As shown by the prototype in line 2, each thread routine takes as input a single generic pointer and returns a generic pointer.
- If you want to pass multiple arguments to a thread routine, then you should put the arguments into a structure and pass a pointer to the structure.
- Similarly, if you want the thread routine to return multiple arguments, you can return a pointer to a structure.
- Line4 marks the beginning of the code for the main thread.
- The main thread declares a single local variable tid, which will be used to store the thread ID of the peer thread(line6).
- The main thread creates a new peer thread by calling the pthread_create function(line7).
- When the call to pthread_create returns, the main thread and the newly created peer thread are running concurrently, and tid contains the ID of the new thread.
- The main thread waits for the peer thread to terminate with the call to pthread_join in line 8.
- Finally, the main thread calls exit (line9), which terminates all threads(in this case just the main thread) currently running in the process.
- Lines 12–16 define the thread routine for the peer thread. It simply prints a string and then terminates the peer thread by executing the return statement in line 15.

Posix Threads

- Posix threads (Pthreads) is a standard interface for manipulating threads from C programs.
- It was adopted in 1995 and is available on most Unix systems.
- Pthreads defines about 60 functions that allow programs to create, kill, and reap threads, to share data safely with peer threads, and to notify peers about changes in the system state.

```
/* hello.c: The Pthreads "Hello, world!" program */
void *thread(void *vargp);

int main()
{
        pthread_t   tid;
        pthread_create(&tid, NULL, thread, NULL);
        pthread_join(tid, NULL);
        exit(0);
}
void *thread(void *vargp) /* Thread routine */
{
   printf("Hello, world!\n");
     return NULL;
}
```

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

WEEK 14 Lecture 1

Concurrent Programming

THREADS SYNCHRONIZATION

Shared Variables in Threaded C Programs

- **Question: Which variables in a threaded C program are shared?**
  - **The answer is not as simple as**

    "*global variables are shared*" and

    "*stack variables are private*"

Def: **A variable `x` is *shared* if and only if multiple threads reference some instance of `x`.**

- Requires answers to the following questions:
  - **What is the memory model for threads?**
  - **How are instances of variables mapped to memory?**
  - **How many threads might reference each of these instances?**

Threads Memory Model

- **Conceptual model:**
  - Multiple threads run within the context of a single process
  - Each thread has its own separate thread context
    - Thread ID, stack, stack pointer, PC, condition codes, and GP registers
  - All threads share the remaining process context
    - Code, data, heap, and shared library segments of the process virtual address space
    - Open files and installed handlers
- **Operationally, this model is not strictly enforced:**
  - Register values are truly separate and protected, but…
  - Any thread can read and write the stack of any other thread

The mismatch between the conceptual and operation model is a source of confusion and errors

Example Program to Illustrate Sharing

```
void *thread(void *vargp);
char **ptr;  /* global var */

int main()
{
    long i;
    pthread_t  tid[2];
    char *msgs[2] = {
        "Hello from foo",
        "Hello from bar"
    };

    ptr = msgs;
    for (i = 0; i < 2; i++)
     pthread_create(&tid[i], NULL, thread, (void *)i);
        pthread_exit(NULL);
}
```

sharing.c

```
void *thread(void *vargp)
{
    long myid = (long)vargp;
    static int cnt = 0;

    printf("[%ld]:  %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
    return NULL;
}
```

Peer threads reference main thread's stack indirectly through global ptr variable

The example program consists of a main thread that creates two peer threads.
The main thread passes a unique ID to each peer thread, which uses the ID to print a personalized message, along with a count of the total number of times that the thread routine has been invoked.

Mapping Variable Instances to Memory

- Global variables
  - *Def:* Variable declared outside of a function
  - **Virtual memory contains exactly one instance of any global variable**
- Local variables
  - *Def:* Variable declared inside function without `static` attribute
  - **Each thread stack contains one instance of each local variable**
- Local static variables
  - *Def:* Variable declared inside function with the `static` attribute
  - **Virtual memory contains exactly one instance of any local static variable.**

Mapping Variable Instances to Memory

Global var: 1 instance (`ptr [data]`)

Local vars: 1 instance (`i.m, msgs.m,tid.m`)

Local var: 2 instances (
`myid.p0` [peer thread 0's stack],
`myid.p1` [peer thread 1's stack]
)

```
char **ptr;  /* global var */

int main()
{
   long i;
   pthread_t tid[2];
   char *msgs[2] ={
      "Hello from foo",
      "Hello from bar"
   };

   ptr = msgs;
   for (i = 0; i < 2; i++)
      Pthread_create(&tid[i],  NULL, thread, (void
*)i);
   Pthread_exit(NULL);
}
```

sharing.c

```
void *thread(void *vargp)
{
   long myid = (long)vargp;
   static int cnt = 0;

   printf("[%ld]:  %s (cnt=%d)\n",
      myid, ptr[myid], ++cnt);
   return NULL;
}
```

Local static var: 1 instance (`cnt [data]`)

Shared Variable Analysis

- Which variables are shared?

Variable instance	*Referenced by main thread?*	*Referenced by peer thread 0?*	*Referenced by peer thread 1?*
`ptr`	yes	yes	yes
`cnt`	no	yes	yes
`i.m`	yes	no	no
`msgs.m`	yes	yes	yes
`tid.m`	yes	no	no
`myid.p0`	no	yes	no
`myid.p1`	no	no	yes

```
char **ptr;  /* global var */
int main()
{
   long i;
   pthread_t tid[2];
   char *msgs[2] = {
      "Hello from foo",
      "Hello from bar"
   };
   ptr = msgs;
   for (i = 0; i < 2; i++)
      Pthread_create(&tid[I],
      NULL, thread, (void *)i);
   Pthread_exit(NULL);
}
```

- Answer: A variable `x` is shared iff multiple threads reference at least one instance of `x`.
- Thus:
  - **`ptr`, `cnt`, and `msgs` are shared**
  - **`i`, tid, and `myid` are *not* shared**

```
void *thread(void *vargp)
{
   long myid = (long)vargp;
   static int cnt = 0;
   printf("[%ld]:  %s (cnt=%d)\n",
      myid, ptr[myid], ++cnt);
   return NULL;
}
```

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 12 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - **Randal E. Bryant and David R. O'Hallaron**
    3rd Edition, Global Edition Prentice Hall 2016
    ISBN 10:1-292-10176-8

SOC 2040
SYSTEM PROGRAMMING

CONCURRENT PROGRAMMING

Concurrent Programming
THREADS SYNCHRONIZATION

Synchronizing Threads

- **Shared variables are handy...**
- **…but introduce the possibility of nasty *synchronization* errors.**

badcnt.c: Improper Synchronization

- Shared variables can be convenient, but they introduce the possibility of nasty synchronization errors.
- Consider the badcnt.c program in Figure, which creates two threads, each of which increments a global shared counter variable called cnt.
- Since each thread increments the counter niters times, we expect its final value to be 2×niters.
- This seems quite simple and straight forward.
- However, when we run badcnt.c on our Linux system, we not only get wrong answers, we get different answers each time!

```c
/* Global shared variable */
volatile long cnt = 0; /* Counter */

int main(int argc, char **argv)
{
   long niters;
   pthread_t tid1, tid2;

   niters = atoi(argv[1]);
   pthread_create(&tid1, NULL, thread, &niters);
   pthread_create(&tid2, NULL, thread, &niters);
   pthread_join(tid1, NULL);
   pthread_join(tid2, NULL);

   /* Check result */
   if (cnt != (2 * niters))
      printf("BOOM! cnt=%ld\n", cnt);
   else
      printf("OK cnt=%ld\n", cnt);
   exit(0);
}
```

badcnt.c

```c
/* Thread routine */
void *thread(void *vargp)
{
   long i, niters =    *((long *)vargp);

   for (i = 0; i < niters; i++)
      cnt++;

   return NULL;
}
```

```
$./badcnt 10000
OK cnt=20000
$./badcnt 10000
BOOM! cnt=13051
$
```

`cnt` should equal 20,000.

What went wrong?

Assembly Code for Counter Loop

So what went wrong? To understand the problem clearly, we need to study the assembly code for the counter loop, as shown in Figure. We will find it helpful to partition the loop code for thread i into five parts:

C code for counter loop in thread i

```
for (i = 0; i < niters; i++)
    cnt++;
```

thread1 thread2

Cnt++ Li Ui Si

Critical region

Asm code for thread i

```
    movq   (%rdi), %rcx
    testq  %rcx,%rcx
    jle    .L2
    movq   $0, %rax
.L3:
    movq   cnt(%rip),%rdx
    addq   $1, %rdx
    movq   %rdx, cnt(%rip)
    addq   $1, %rax
    cmpq   %rcx, %rax
    jne    .L3
.L2:
```

%rdi =&niters

%rcx =niters

H_i : Head

%rdx =cnt

L_i : Load cnt

U_i : Update cnt

S_i : Store cnt

T_i : Tail

Synchronizing Threads

- To understand the problem clearly, we need to study the assembly code for the counter loop as shown in Figure.
- We will find it helpful to partition the loop code for thread i into five parts:

  Hi: The block of instructions at the head of the loop

  Li: The instruction that loads the shared variable cnt into register %eaxi, where %eaxi denotes the value of register %eax in thread i

  Ui: The instruction that updates (increments) %eaxi

  Si: The instruction that stores the updated value of %eaxi back to the shared variable cnt

  Ti: The block of instructions at the tail of the loop

- Notice that the head and tail manipulate only local stack variables, while Li, Ui, and Si manipulate the contents of the shared counter variable.
- When the two peer threads in badcnt.c run concurrently on a uniprocessor, the machine instructions are completed one after the other in some order.
- Thus, each concurrent execution defines some total ordering (or interleaving) of the instructions in the two threads.
- Unfortunately, some of these orderings will produce correct results, but others will not.

Concurrent Execution

- ***Key idea:*** **In general, any sequentially consistent interleaving is possible, but some give an unexpected result!**
  - **I_i denotes that thread i executes instruction I**
  - **$\%rdx_i$ is the content of %rdx in thread i's context**

	i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
Step 1	1	H_1	-	-	0
Step 2	1	L_1	0	-	0
Step 3	1	U_1	1	-	0
Step 4	1	S_1	1	-	1
Step 5	2	H_2	-	-	1
Step 6	2	L_2	-	1	1
Step 7	2	U_2	-	2	1
Step 8	2	S_2	-	2	2
Step 9	2	T_2	-	2	2
Step 10	1	T_1	1	-	2

Thread 1 critical section

Thread 2 critical section

OK

In general, there is no way for you to predict whether the operating system will choose a correct ordering for your threads. For example, Figure shows the step-by-step operation of a correct instruction ordering. After each thread has updated the shared variable cnt, its value in memory is 2, which is the expected result.

Concurrent Execution (cont)

- Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2

	i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
Step 1	1	H_1	-	-	0
Step 2	1	L_1	0	-	0
Step 3	1	U_1	1	-	0
Step 4	2	H_2	-	-	0
Step 5	2	L_2	-	0	0
Step 6	1	S_1	1	-	1
Step 7	1	T_1	1	-	1
Step 8	2	U_2	-	1	1
Step 9	2	S_2	-	1	1
Step 10	2	T_2	-	1	1

Oops!

On the other hand, the ordering in Figure above produces an incorrect value for cnt. The problem occurs because thread 2 loads cnt in step 5, after thread 1 loads cnt in step 2, but before thread 1 stores its updated value in step 6. Thus, each thread ends up storing an updated counter value of 1.

Concurrent Execution (cont)

- How about this ordering?

	i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
Step 1	1	H_1			0
Step 2	1	L_1	0		
Step 3	2	H_2			
Step 4	2	L_2		0	
Step 5	2	U_2		1	
Step 6	2	S_2		1	1
Step 7	1	U_1	1		
Step 8	1	S_1	1		1
Step 9	1	T_1			1
Step 10	2	T_2			1

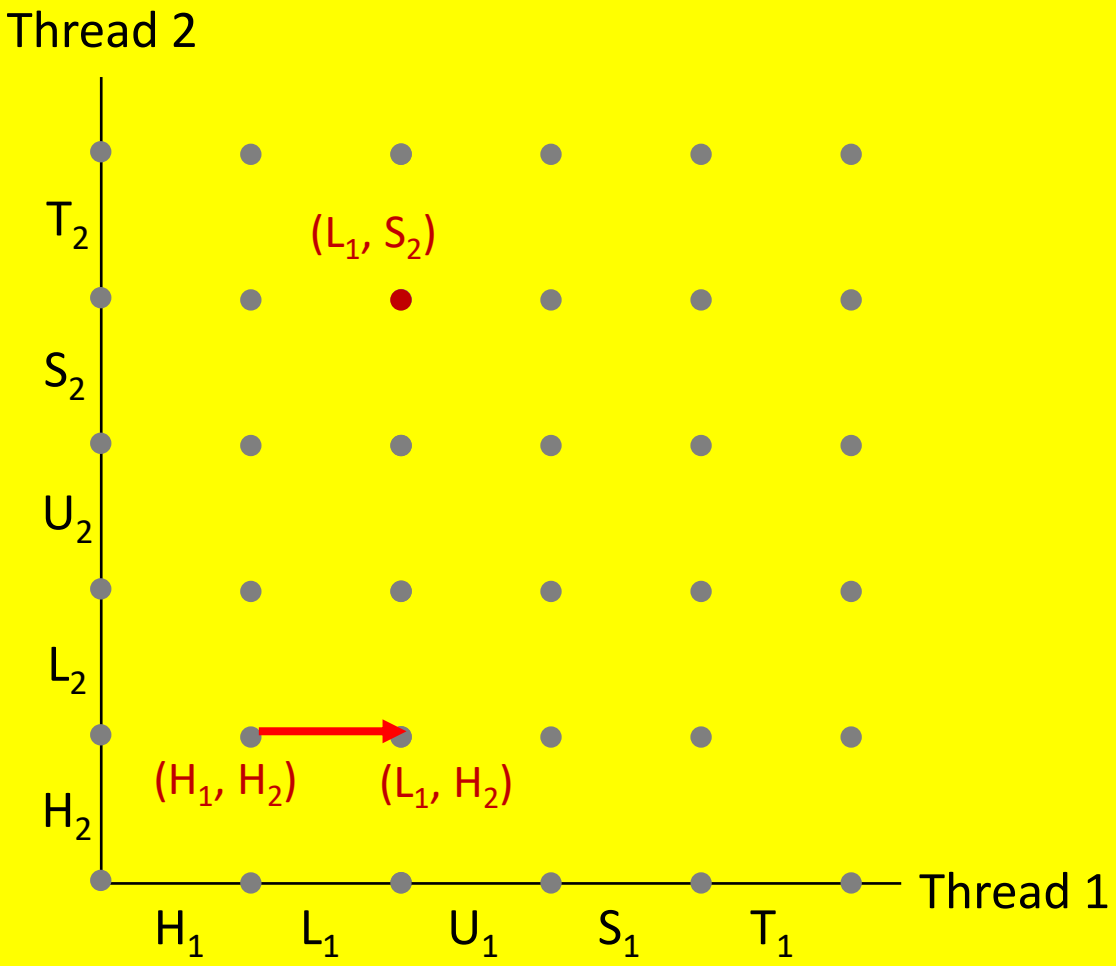
Oops!

- **These notions of correct and incorrect instruction orderings can be understood with the help of a device known as a progress graph,**
- We can analyze the behavior using a *progress graph*

Progress Graphs

- A progress graph models the execution of n concurrent threads as a trajectory through an n-dimensional Cartesian space.
- Each axis k corresponds to the progress of thread k.
- Each point (I1,I2,...,In) represents the state where thread k (k=1 ,...,n) has completed instruction Ik.
- The origin of the graph corresponds to the initial state where none of the threads has yet completed an instruction.
- Figure shows the two-dimensional progress graph for the first loop iteration of the badcnt.c program.
- The horizontal axis corresponds to thread 1, the vertical axis to thread2.
- Point(L1,S2) corresponds to the state where thread1 has completed L1 and thread 2 has completed S2.
- A progress graph models instruction execution as a transition from one state to another.
- A transition is represented as a directed edge from one point to an adjacent point.
- Legal transitions move to the right (an instruction in thread 1 completes) or up (an instruction in thread 2 completes).
- Two instructions cannot complete at the same time—diagonal transitions are not allowed.
- Programs never run backwards, so transitions that move down or to the left are not legal either.

Progress Graphs



A *progress graph* depicts the discrete *execution state space* of concurrent threads.

Each axis corresponds to the sequential order of instructions in a thread.

Each point corresponds to a possible *execution state* ($Inst_1$, $Inst_2$).

E.g., (L_1, S_2) denotes state where thread 1 has completed L_1 and thread 2 has completed S_2.

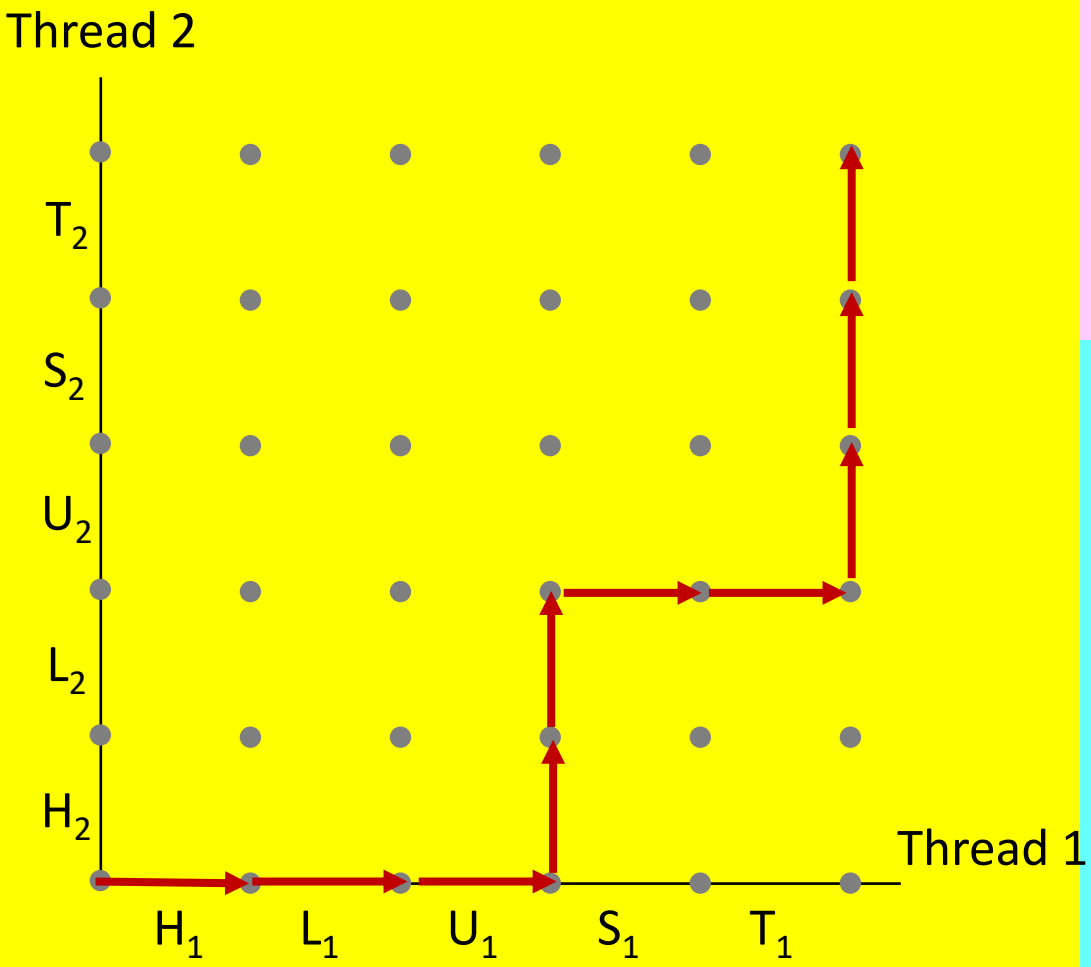
Progress Graphs

- The execution history of a program is modeled as a trajectory through the state space.
- Figure shows the trajectory that corresponds to the following instruction ordering:

$$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$$

- For thread i, the instructions (Li, Ui,Si) that manipulate the contents of the shared variable cnt constitute a critical section (with respect to shared variable cnt) that should not be interleaved with the critical section of the other thread.
- In other words, we want to ensure that each thread has mutually exclusive access to the shared variable while it is executing the instructions in its critical section.
- The phenomenon in general is known as mutual exclusion.

Trajectories in Progress Graphs



A *trajectory* is a sequence of legal state transitions that describes one possible concurrent execution of the threads.

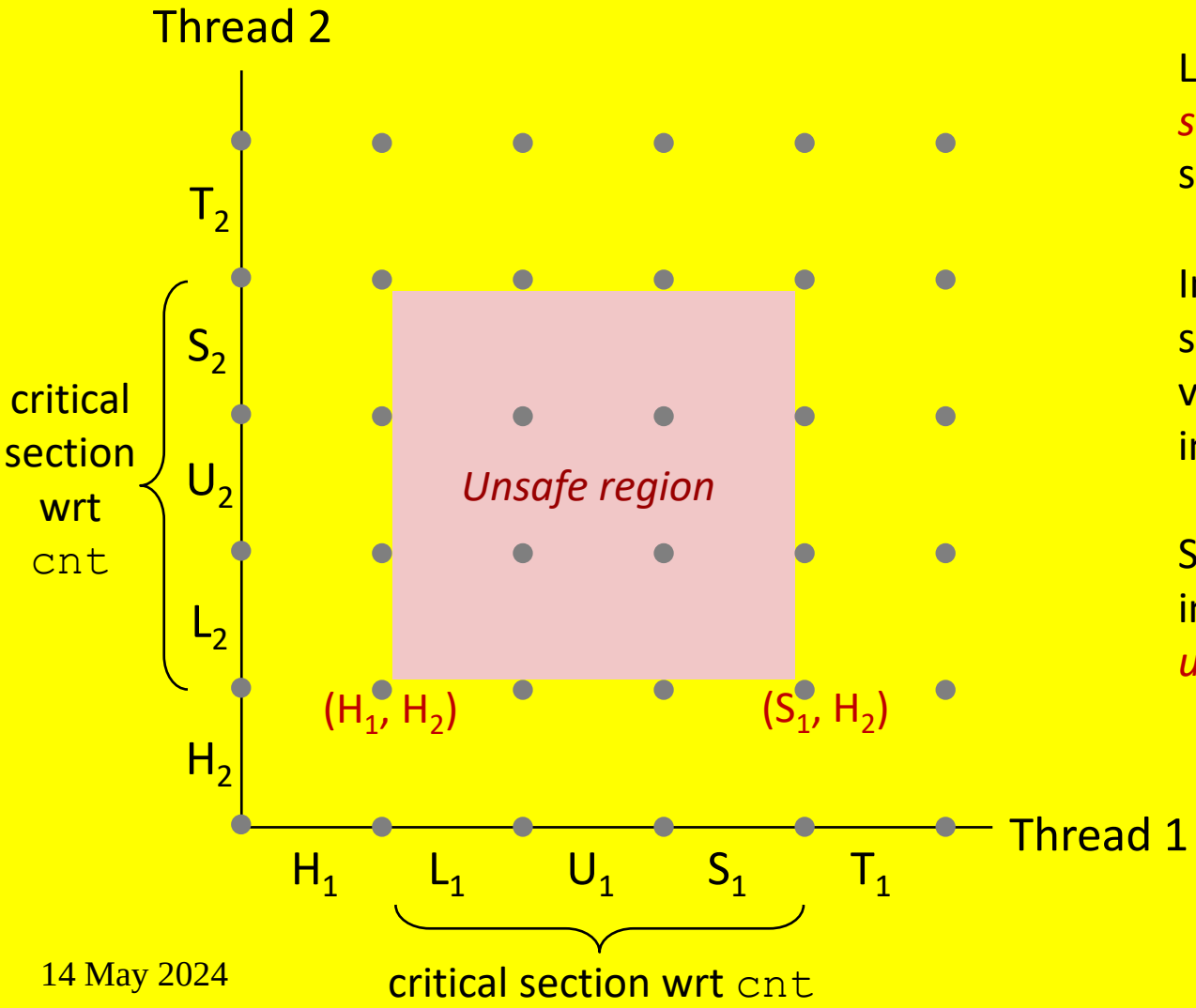
Example:

H1, L1, U1, H2, L2, S1, T1, U2, S2, T2

Progress Graphs

- On the progress graph, the intersection of the two critical sections defines a region of the state space known as an unsafe region.
- Figure shows the unsafe region for the variable cnt.
- Notice that the unsafe region abuts, but does not include, the states along its perimeter.
- For example, states (H1,H2) and (S1,H2) abut the unsafe region, but are not part of it.
- A trajectory that skirts the unsafe region is known as a safe trajectory.
- Conversely, a trajectory that touches any part of the unsafe region is an unsafe trajectory.
- Figure shows examples of safe and unsafe trajectories through the state space of our example badcnt.c program.
- The upper trajectory skirts the unsafe region along its left and top sides, and thus is safe.
- The lower trajectory crosses the unsafe region, and thus is unsafe.
- Any safe trajectory will correctly update the shared counter.
- In order to guarantee correct execution of our example threaded program—and indeed any concurrent program that shares global data structures—we must somehow synchronize the threads so that they always have a safe trajectory.

Critical Sections and Unsafe Regions

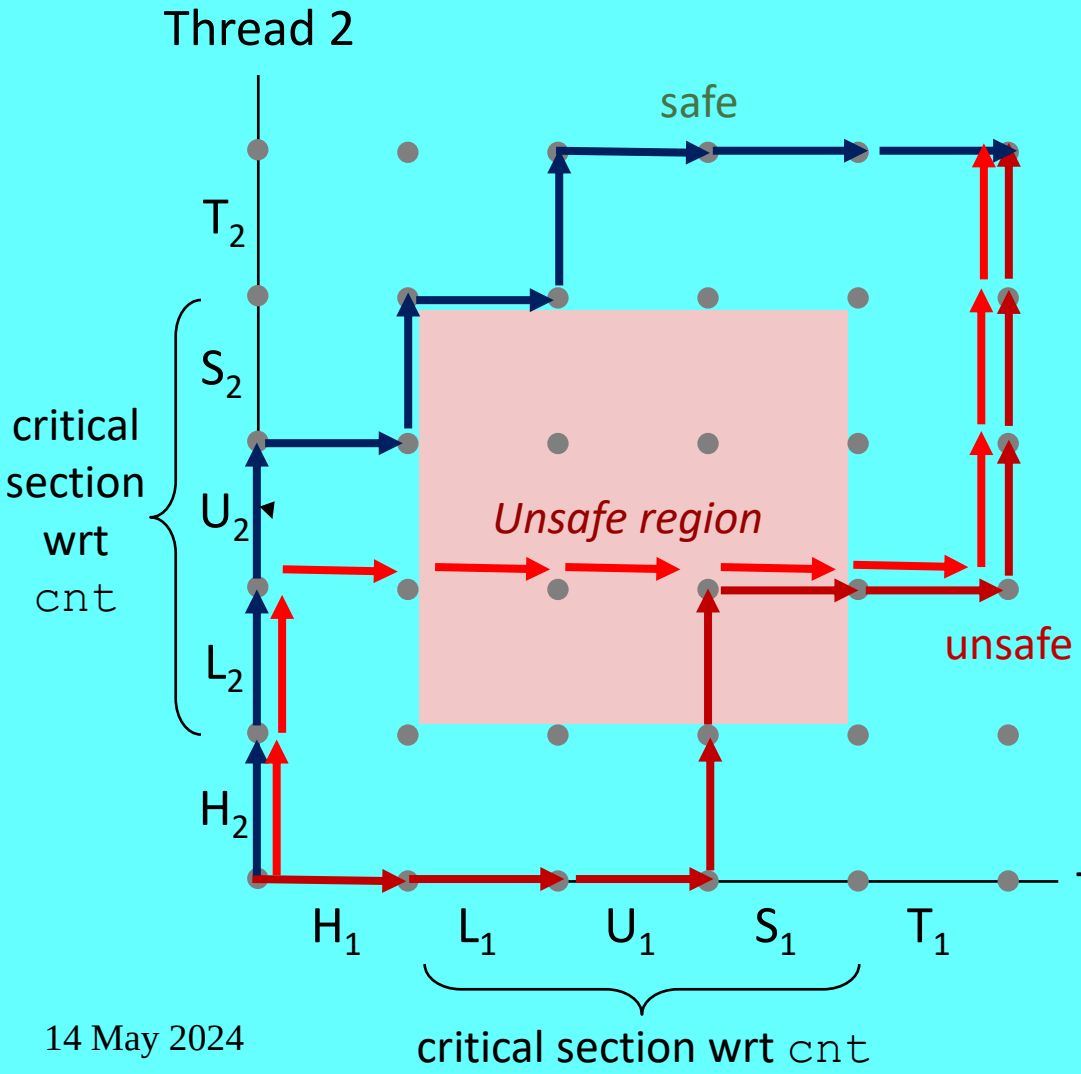


L, U, and S form a *critical section* with respect to the shared variable `cnt`

Instructions in critical sections (wrt some shared variable) should not be interleaved

Sets of states where such interleaving occurs form *unsafe regions*

Critical Sections and Unsafe Regions



Def: A trajectory is *safe* iff it does not enter any unsafe region

Claim: A trajectory is correct (wrt `cnt`) iff it is safe

Using the progress graph in Figure , classify the following trajectories as either safe or unsafe.

A.
H1,L1,U1,H2,L2,S1,T1,U2,S2,T2
Usafe

B.
H2,L2,H1,L1,U1,S1,T1,U2,S2,T2
Unsafe

C.
H2,L2,U2,H1,S2,L1,T2,U1,S1,T1
Safe

Enforcing Mutual Exclusion

- ***Question:*** **How can we guarantee a safe trajectory?**

- Answer: We must ***synchronize*** the execution of the threads so that they can never have an unsafe trajectory.
  - i.e., need to guarantee ***mutually exclusive access*** for each critical section.

- Classic solution:
  - Semaphores (Edsger Dijkstra)

- Other approaches
  - Mutex and condition variables (Pthreads)
  - Monitors (Java)

Semaphores

- ***Semaphore:***
  - Edsger Dijkstra, a pioneer of concurrent programming, proposed a classic solution to the problem of synchronizing different execution threads based on a special type of variable called a semaphore.
  - **A semaphore, s, is a global variable with a nonnegative integer value that can only be manipulated by two special operations, called P and V:**

P(s) : If s is nonzero, then P decrements s and returns immediately.

If s is zero, then suspend the thread until s becomes nonzero and the process is restarted by a V operation.

After restarting, the P operation decrements s and returns control to the caller.

V (s): The V operation increments s by 1.

If there are any threads blocked at a P operation waiting for s to become nonzero, then the V operation restarts exactly one of these threads, which then completes its P operation by decrementing s.

Semaphores

- ***Semaphore:***

- **The test and decrement operations in P occur indivisibly, in the sense that once the semaphore s becomes nonzero, the decrement of s occurs without interruption.**
- **The increment operation in V also occurs indivisibly, in that it loads, increments, and stores the semaphore without interruption.**
- **Notice that the definition of V does not define the order in which waiting threads are restarted.**
- **The only requirement is that the V must restart exactly one waiting thread.**
- **Thus, when several threads are waiting at a semaphore, you cannot predict which one will be restarted as a result of the V.**
- **The definitions of P and V ensure that a running program can never enter a state where a properly initialized semaphore has a negative value.**
- **This property, known as the semaphore invariant, provides a powerful tool for controlling the trajectories of concurrent programs**

Semaphores

- ***Semaphore:***
  - non-negative global integer synchronization variable.
  - Manipulated by *P* and *V* operations.

- P(s)
  - If *s* is nonzero, then decrement *s* by 1 and return immediately.
    - Test and decrement operations occur atomically (indivisibly)
  - If *s* is zero, then suspend thread until *s* becomes nonzero and the thread is restarted by a V operation.
  - After restarting, the P operation decrements *s* and returns control to the caller.
- ***V(s):***
  - Increment *s* by 1.
    - Increment operation occurs atomically
  - If there are any threads blocked in a P operation waiting for *s* to become non-zero, then restart exactly one of those threads, which then completes its P operation by decrementing *s*.
- Semaphore invariant: *(s >= 0)*

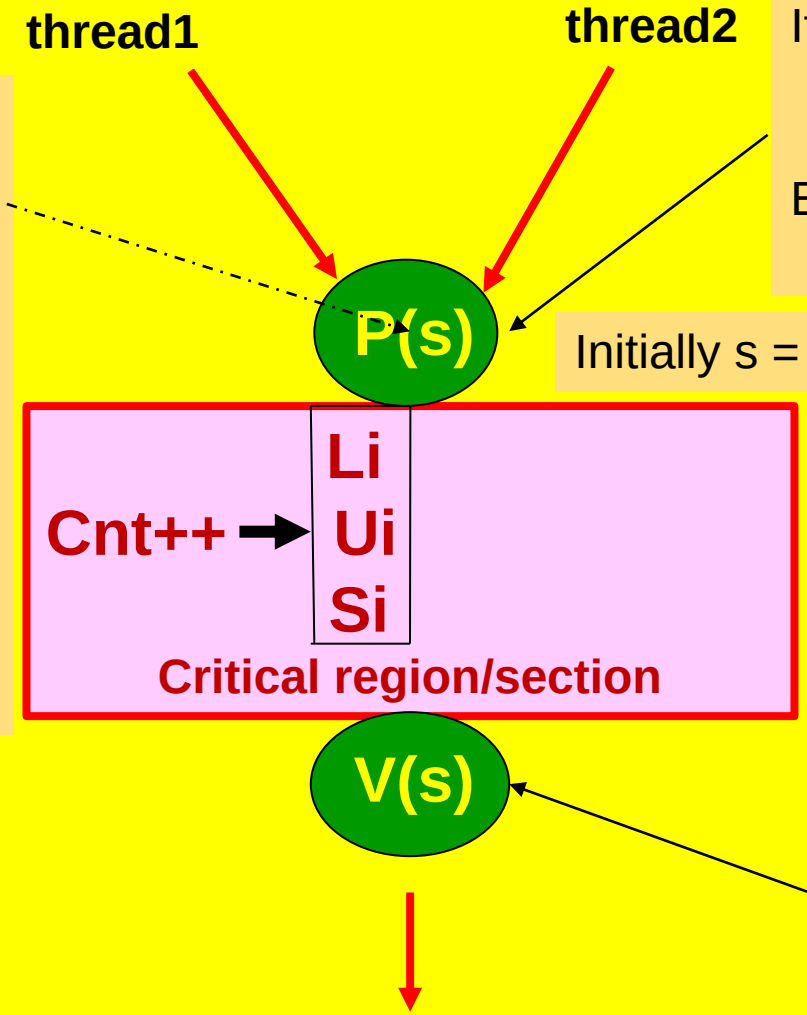
Using Semaphores for Mutual Exclusion

- **Basic idea:**
  - **Associate a unique semaphore *mutex*, initially 1, with each shared variable (or related set of shared variables).**
  - **Surround corresponding critical sections with *P(mutex)* and *V(mutex)* operations.**
- **Terminology:**
  - ***Binary semaphore*: semaphore whose value is always 0 or 1**
  - ***Mutex:* binary semaphore used for mutual exclusion**
    - **P operation: "locking" the mutex**
    - **V operation: "unlocking" or "releasing" the mutex**
    - ***"Holding"* a mutex: locked and not yet unlocked.**
  - ***Counting semaphore*: used as a counter for set of available resources.**

Using Semaphores for Mutual Exclusion

- Semaphores provide a convenient way to ensure mutually exclusive access to shared variables.
- The basic idea is to associate a semaphore s, initially 1, with each shared variable (or related set of shared variables) and then surround the corresponding critical section with P(s)and V (s)operations.
- A semaphore that is used in this way to protect shared variables is called a binary semaphore because its value is always 0 or 1.
- Binary semaphores whose purpose is to provide mutual exclusion are often called mutexes.
- Performing a P operation on a mutex is called locking the mutex. Similarly, performing the V operation is called unlocking the mutex.
- A thread that has locked but not yet unlocked a mutex is said to be holding the mutex.
- A semaphore that is used as a counter for a set of available resources is called a counting semaphore.

Using Semaphores for Mutual Exclusion



s mutex is used as a lock

s=1 means lock is open

s=0 means lock is closed

P(s) – Lock mutex
If s==1 then
s=s -1 (decrement s)
Enter Critical region/section
Else
Wait or block (until s becomes 1)

Initially s = 1

V(s) - Unlock mutex
s=s + 1 (increment s)

C Semaphore Operations

The Posix standard defines a variety of functions for manipulating semaphores.

Pthreads functions:

```
#include <semaphore.h>

int sem_init(sem_t *s, 0, unsigned int val);} /* s = val */

int sem_wait(sem_t *s);  /* P(s) */
int sem_post(sem_t *s);  /* V(s) */
```

- **The sem_init function initializes semaphore sem to value.**
- **Each semaphore must be initialized before it can be used.**
- **For our purposes, the middle argument is always 0.**
- **Programs perform P and V operations by calling the sem_wait and sem_post functions, respectively.**
- **For conciseness, we prefer to use the following equivalent P and V wrapper functions instead:**

wrapper functions:

```
void P(sem_t *s){sem_wait(&s);} /* Wrapper function for sem_wait */
void V(sem_t *s){sem_post(&s);} /* Wrapper function for sem_post */
```

goodcnt.c: Proper Synchronization

Putting it all together, to properly synchronize the example counter program in Figure using semaphores, we first declare a semaphore called mutex:

- **Define and initialize a mutex for the shared variable `cnt`:**

```
volatile long cnt = 0;  /* Counter */
sem_t mutex;            /* Semaphore that protects cnt */
  then initialize it to unity in the main routine:
sem_init(&mutex, 0, 1); /* mutex = 1 */
```

- **Surround** critical section with *P* and *V*:

Finally, we protect the update of the shared cnt variable in the thread routine by surrounding it with P and V operations:

```
for (i = 0; i < niters; i++)
{
    P(&mutex);
    cnt++;
    V(&mutex);
 }
```

goodcnt.c

When we run the properly synchronized program,it now produces the correct answer each time.

```
$./goodcnt 10000
OK cnt=20000
$./goodcnt 10000
OK cnt=20000
$
```

Warning: It's orders of magnitude slower than `badcnt.c`.

goodcnt.c: Proper Synchronization

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include<semaphore.h>

/* Global shared variable */
volatile long cnt = 0; /* Counter */
sem_t  mutex;
sem_init(&mutex, 0, 1);

int main(int argc, char **argv)
{
   long niters;
   pthread_t tid1, tid2;
   niters = atoi(argv[1]);
   pthread_create(&tid1, NULL, thread, &niters);
   pthread_create(&tid2, NULL, thread, &niters);
   pthread_join(tid1, NULL);
   pthread_join(tid2, NULL);

   /* Check result */
   if (cnt != (2 * niters))
      printf("BOOM! cnt=%ld\n", cnt);
   else
      printf("OK cnt=%ld\n", cnt);
   exit(0);
}
```

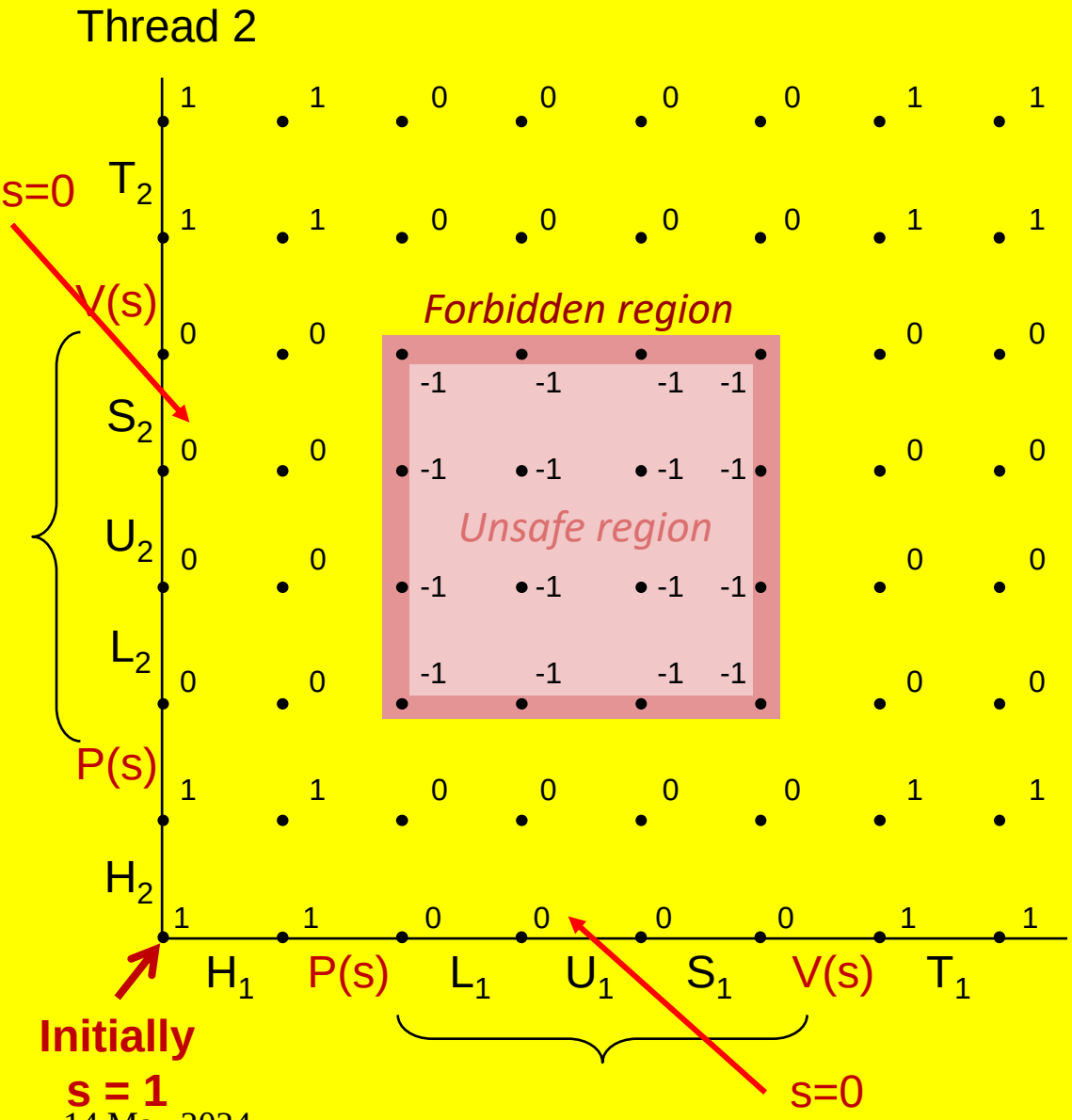
goodcnt.c

```c
/* Thread routine */
void *thread(void *vargp)
{
   long i, niters =  *((long *)vargp);

   for (i = 0; i < niters; i++)
    {
       P(&mutex);
       cnt++;
       V(&mutex);
    }
     return NULL;
}

void P(sem_t *s)
{
   sem_wait(&s);
}

void V(sem_t *s)
{
   sem_post(&s);
}
```

Using Semaphores for Mutual Exclusion

- The progress graph in Figure shows how we would use binary semaphores to properly synchronize our example counter program.
- Each state is labeled with the value of semaphore s in that state.
- The crucial idea is that this combination of P and V operations creates a collection of states, called a forbidden region, where s<0.
- Because of the semaphore invariant, no feasible trajectory can include one of the states in the forbidden region.
- And since the forbidden region completely encloses the unsafe region, no feasible trajectory can touch any part of the unsafe region.
- Thus, every feasible trajectory is safe, and regardless of the ordering of the instructions at run time, the program correctly increments the counter.
- In an operational sense, the forbidden region created by the P and V operations makes it impossible for multiple threads to be executing instructions in the enclosed critical region at any point in time.
- In other words, the semaphore operations ensure mutually exclusive access to the critical region.

Why Mutexes Work



Provide mutually exclusive access to shared variable by surrounding critical section with *P* and *V* operations on semaphore s (initially set to 1)

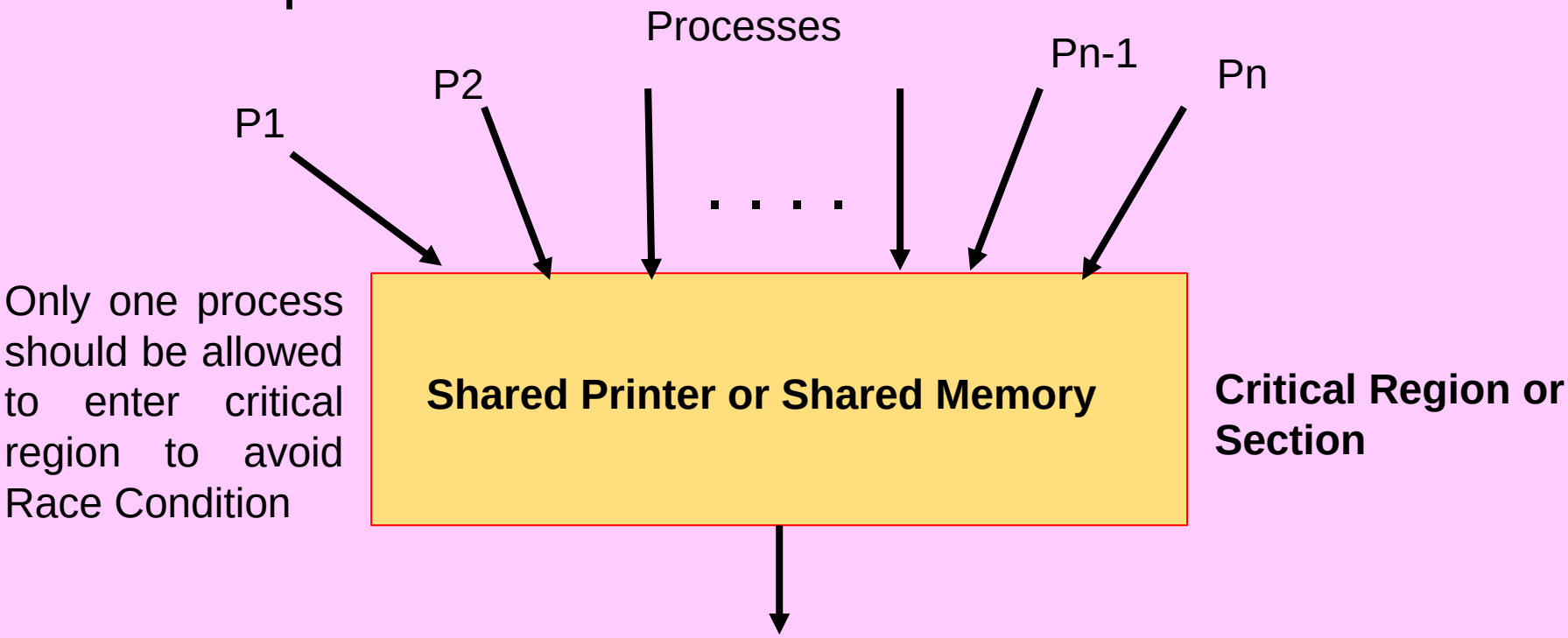
Semaphore invariant creates a *forbidden region* that encloses unsafe region and that cannot be entered by any trajectory.

Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Races***
  - ***Deadlock***
  - ***Livelock / Starvation / Fairness***

Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Races:*** outcome depends on arbitrary scheduling decisions elsewhere in the system
    - Example: who gets the last seat on the airplane?



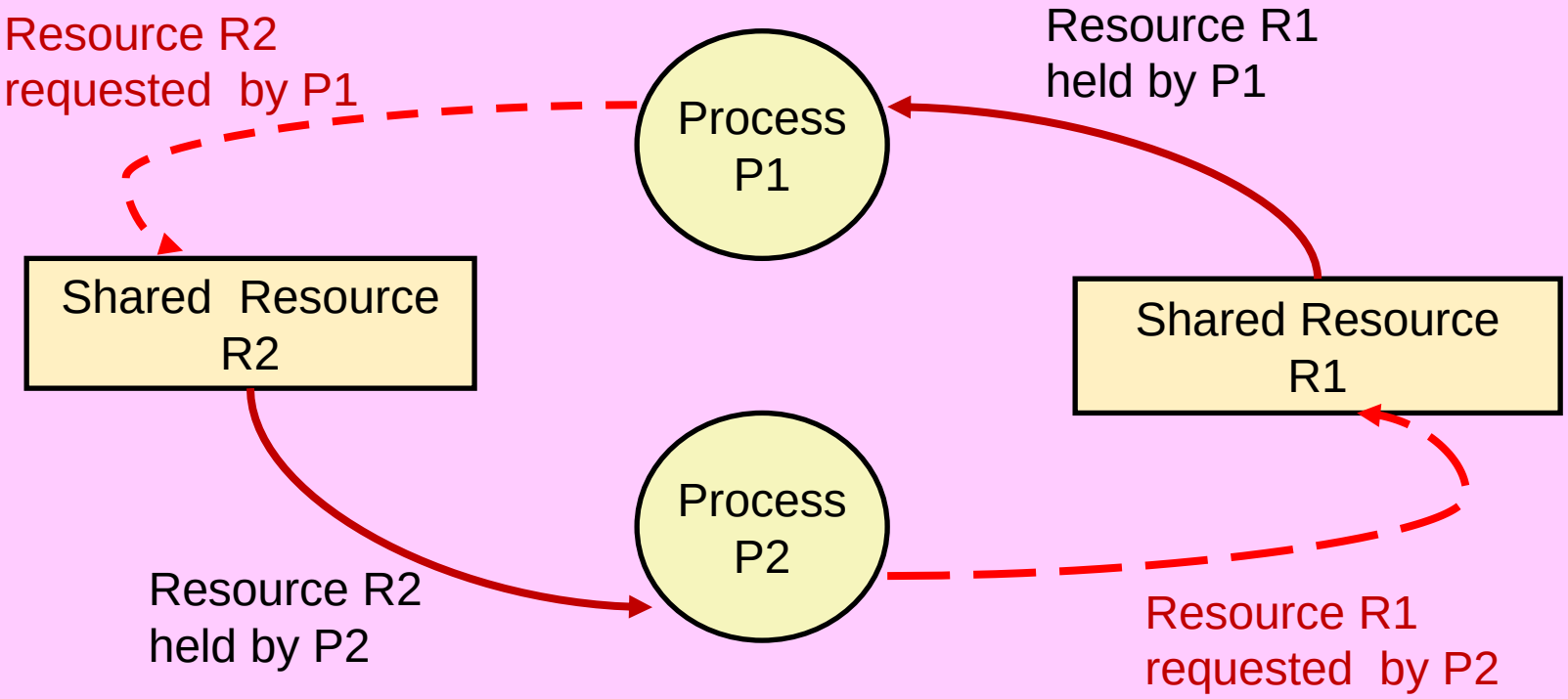
Classical problem classes of concurrent programs:

- **RACE CONDITION**

  - **occurs when more than one process enter the critical region containing shared resource thereby producing incorrect results**

Concurrent Programming is Hard!

- Classical problem classes of concurrent programs:
  - ***Deadlock:*** improper resource allocation prevents forward progress
    - Example: traffic gridlock



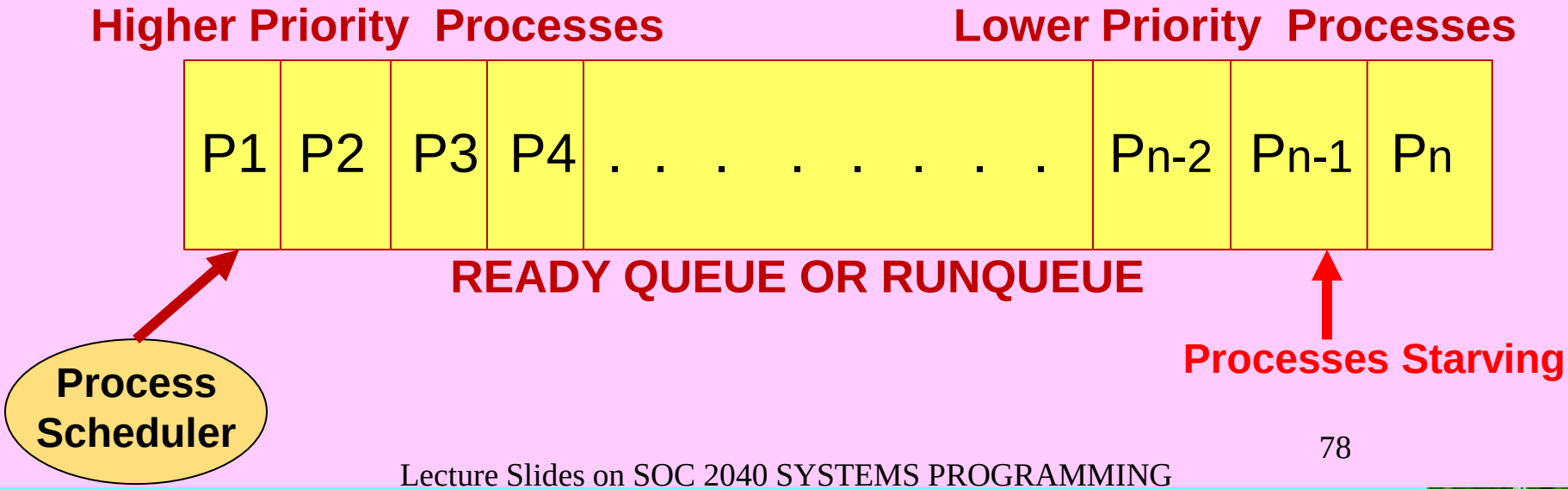
Classical problem classes of concurrent programs:

DEADLOCK

- **is a condition where a set of processes are waiting because each process is holding on to a resource and waiting for some other resource that is being held by another process which is also waiting for the resource held by another resource in a chain.**
- **Hence, no process can go forward as no process is going to release the resource held by it.**

Concurrent Programming is Hard!

- **Classical problem classes of concurrent programs:**
  - ***Livelock / Starvation / Fairness***: external events and/or system scheduling decisions can prevent sub-task progress
    - Example: people always jump in front of you in line or queue



Classical problem classes of concurrent programs:

STARVATION

- occurs when greedy high priority processes make shared resources unavailable for long periods.
- The low-priority processes requiring these resources will have to wait indefinitely and sometimes never get a chance to be serviced.

Classical problem classes of concurrent programs:

LIVELOCK

- occurs when two or more processes continually repeat the same interaction in response to changes in the other processes without doing any useful work.
- These processes are not in the waiting state, and they are running concurrently without progressing forward.

Summary

- Programmers need a clear model of how variables are shared by threads.
- Variables shared by multiple threads must be protected to ensure mutually exclusive access.
- Semaphores are a fundamental mechanism for enforcing mutual exclusion.

Reading Assignments

Chapter 12 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - Randal E. Bryant and David R. O'Hallaron
  3rd Edition, Global Edition Prentice Hall 2016
  ISBN 10:1-292-10176-8

SOC 2040
SYSTEM PROGRAMMING

CONCURRENT PROGRAMMING

Synchronization: Advanced

14 May 2024

Synchronization: Advanced

Review: Semaphores

- ***Semaphore:*** **non-negative global integer synchronization variable. Manipulated by *P* and *V* operations.**

- P(s)
  - **If *s* is nonzero, then decrement *s* by 1 and return immediately.**
  - **If *s* is zero, then suspend thread until *s* becomes nonzero and the thread is restarted by a V operation.**
  - **After restarting, the P operation decrements *s* and returns control to the caller.**

- ***V(s):***
  - **Increment *s* by 1.**
  - **If there are any threads blocked in a P operation waiting for *s* to become non-zero, then restart exactly one of those threads, which then completes its P operation by decrementing *s*.**

- Semaphore invariant: *(s >= 0)*

Using semaphores to protect shared resources via mutual exclusion

- **Basic idea:**
  - **Associate a unique semaphore *mutex*, initially 1, with each shared variable (or related set of shared variables)**
  - **Surround each access to the shared variable(s) with *P(mutex)* and *V(mutex)* operations**

```
mutex = 1

P(mutex)
cnt++
V(mutex)
```

Using Semaphores to Coordinate Access to Shared Resources

- Another important use of semaphores, besides providing mutual exclusion, is to schedule accesses to shared resources

- **Basic idea:**
- In this scenario, a Thread uses a semaphore operation to notify another thread that some condition has become true
  - Use counting semaphores to keep track of resource state and to notify other threads
  - Use mutex to protect access to resource
- Two classical (IPC Problems)and useful examples are :
  - The Producer-Consumer Problem
  - The Readers-Writers Problem

  Another interesting Classical IPC problem is

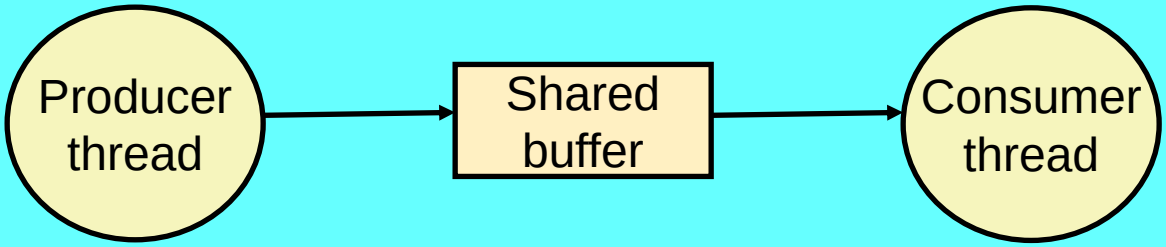
  Dining Philosophers Problem (not of practical use)

Producer-Consumer Problem

- A producer and consumer thread share a bounded buffer with n slots.
- The producer thread repeatedly produces new items and inserts them in the buffer.
- The consumer thread repeatedly removes items from the buffer and then consumes(uses) them.
- Variants with multiple producers and consumers are also possible.
- Since inserting and removing items involves updating shared variables, we must guarantee mutually exclusive access to the buffer.
- But guaranteeing mutual exclusion is not sufficient.
- We also need to schedule accesses to the buffer.
- If the buffer is full (there are no empty slots), then the producer must wait until a slot becomes available.
- Similarly, if the buffer is empty (there are no available items), then the consumer must wait until an item becomes available.

- Producer-consumer interactions occur frequently in real systems.
- For example, in a multimedia system, the producer might encode video frames while the consumer decodes and renders them on the screen.
- The purpose of the buffer is to reduce jitter in the video stream caused by data-dependent differences in the encoding and decoding times for individual frames.
- The buffer provides a reservoir of slots to the producer and a reservoir of encoded frames to the consumer.
- Another common example is the design of graphical user interfaces. The producer detects mouse and keyboard events and inserts them in the buffer.
- The consumer removes the events from the buffer in some priority-based manner and paints the screen.

Producer-Consumer Problem



- **Common synchronization pattern:**
  - **Producer waits for empty *slot*, inserts item in buffer, and notifies consumer**
  - **Consumer waits for *item*, removes it from buffer, and notifies producer**

- **Examples**
  - **Multimedia processing:**
    - **Producer creates MPEG video frames, consumer renders them**
  - **Event-driven graphical user interfaces**
    - **Producer detects mouse clicks, mouse movements, and keyboard hits and inserts corresponding events in buffer**
    - **Consumer retrieves events from buffer and paints the display**

Producer-Consumer on an *n*-element Buffer

- Requires a mutex and two counting semaphores:
  - `mutex`: enforces mutually exclusive access to the the buffer
  - `slots`: counts the available slots in the buffer
  - `items`: counts the available items in the buffer
- Implemented using a shared buffer package called `sbuf`.

sbuf Package - Declarations

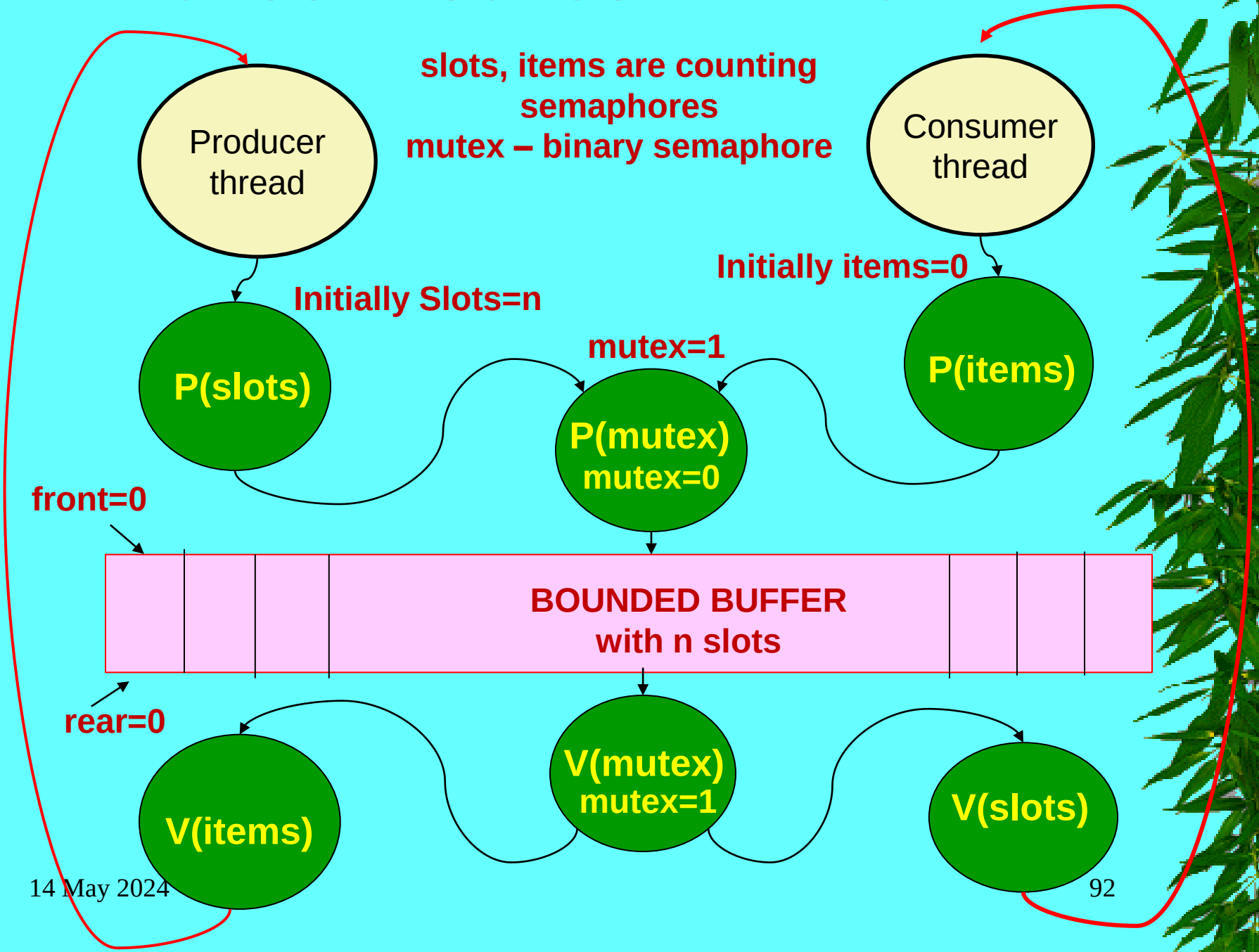
```
typedef struct {
    int *buf;          /* Buffer array */
    int n;             /* Maximum number of slots */
    int front;         /* buf[(front+1)%n] is first item */
    int rear;          /* buf[rear%n] is last item */
    sem_t mutex;       /* Protects accesses to buf */
    sem_t slots;       /* Counts available slots */
    sem_t items;       /* Counts available items */
} sbuf_t;
```

```
void sbuf_init(sbuf_t *sp, int n);
void sbuf_deinit(sbuf_t *sp);
void sbuf_insert(sbuf_t *sp, int item);
int sbuf_remove(sbuf_t *sp);
```

Initializing and deinitializing a shared buffer:

```
/* Create an empty, bounded, shared FIFO buffer with n slots */
void sbuf_init(sbuf_t *sp, int n)
{
    sp->buf = calloc(n, sizeof(int));
    sp->n = n;                       /* Buffer holds max of n items */
    sp->front = sp->rear = 0;        /* Empty buffer iff front == rear */
    sem_init(&sp->mutex, 0, 1);      /* Binary semaphore for locking */
    sem_init(&sp->slots, 0, n);      /* Initially, buf has n empty slots */
    sem_init(&sp->items, 0, 0);      /* Initially, buf has 0 items */
}

/* Clean up buffer sp */
void sbuf_deinit(sbuf_t *sp)
{
    free(sp->buf);
}
```

PRODUCER CONSUMER PROBLEM



sbuf Package - Implementation

Inserting an item into a shared buffer:

PRODUCER THREAD FUNCTION

```
/* Insert item onto the rear of shared buffer sp */
void sbuf_insert(sbuf_t *sp, int item)
{
    P(&sp->slots);                  /* Wait for available slot */
    P(&sp->mutex);                  /* Lock the buffer */
    sp->buf[(++sp->rear)%(sp->n)] = item;  /* Insert the item */
    V(&sp->mutex);                  /* Unlock the buffer */
    V(&sp->items);                  /* Announce available item */
}
```

sbuf.c

Removing an item from a shared buffer:

CONSUMER THREAD FUNCTION

```
/* Remove and return the first item from buffer sp */
int sbuf_remove(sbuf_t *sp)
{
    int item;
    P(&sp->items);                  /* Wait for available item */
    P(&sp->mutex);                  /* Lock the buffer */
    item = sp->buf[(++sp->front)%(sp->n)]; /* Remove the item */
    V(&sp->mutex);                  /* Unlock the buffer */
    V(&sp->slots);                  /* Announce available slot */
    return item;
}
```

Readers-Writers Problem

- The readers-writers problem is a generalization of the mutual exclusion problem.
- A collection of concurrent threads are accessing a shared object such as a data structure in main memory or a database on disk.
- Some threads only read the object, while others modify it.
- Threads that modify the object are called writers.
- Threads that only read it are called readers.
- Writers must have exclusive access to the object, but readers may share the object with an unlimited number of other readers.

Readers-Writers Problem

- In general, there are an unbounded number of concurrent readers and writers.
- Readers-writers interactions occur frequently in real systems.
- For example, in an online airline reservation system, an unlimited number of customers are allowed to concurrently inspect these at assignments, but a customer who is booking a seat must have exclusive access to the database.
- As another example, in a multithreaded caching Web proxy, an unlimited number of threads can fetch existing pages from the shared page cache, but any thread that writes a new page to the cache must have exclusive access.

Readers-Writers Problem

- The readers-writers problem has several variations, each based on the priorities of readers and writers.
- The first readers-writers problem, which favors readers, requires that no reader be kept waiting unless a writer has already been granted permission to use the object.
- In otherwords, no reader should wait simply because a writer is waiting.
- The second readers-writers problem, which favors writers, requires that once a writer is ready to write, it performs its write as soon as possible.
- Unlike the first problem, a reader that arrives after a writer must wait, even if the writer is also waiting.

Readers-Writers Problem

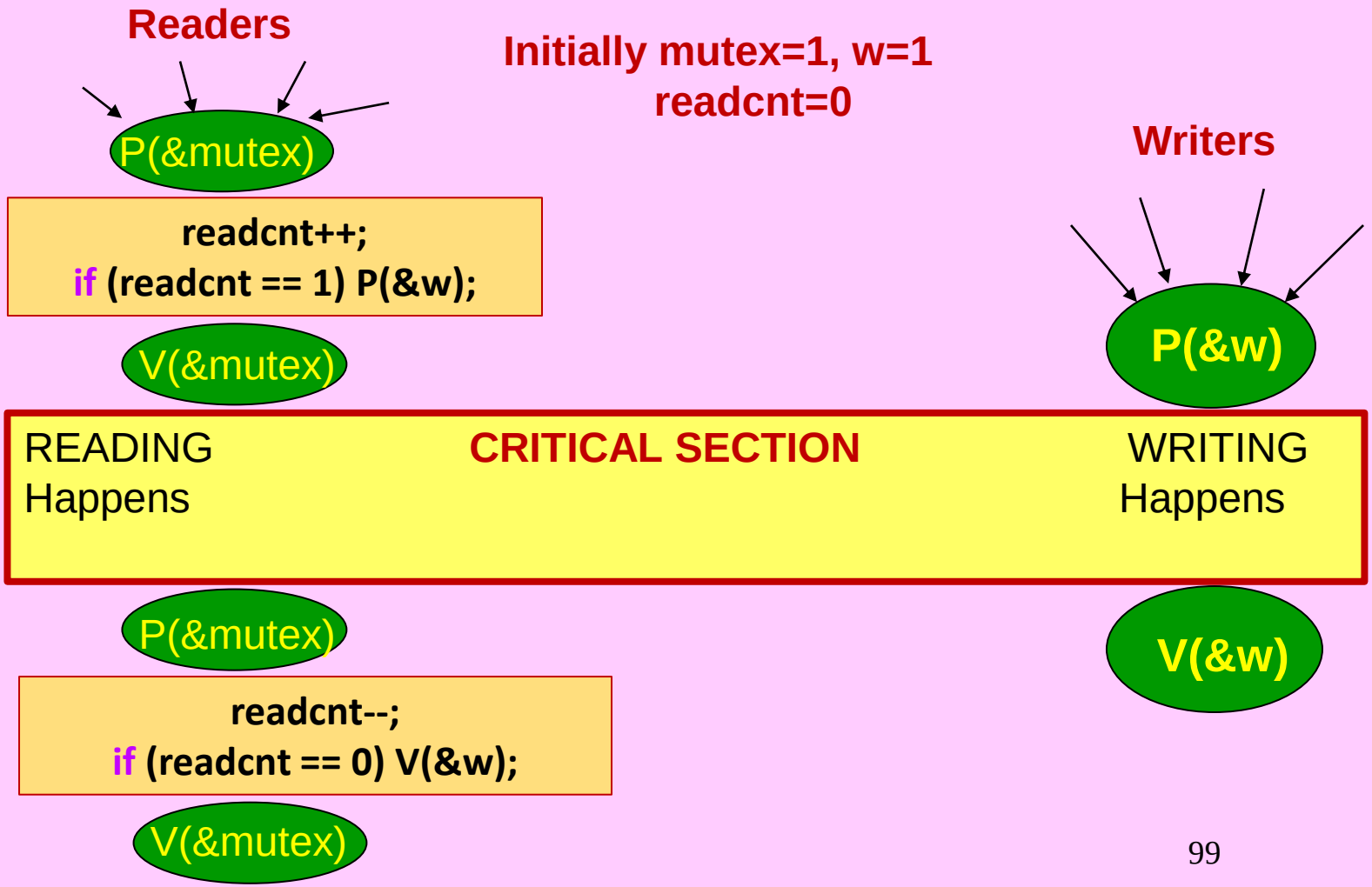
- Generalization of the mutual exclusion problem
- Problem statement:
  - *Reader* threads only read the object
  - *Writer* threads modify the object
  - Writers must have exclusive access to the object
  - Unlimited number of readers can access the object
- Occurs frequently in real systems, e.g.,
  - Online airline reservation system
  - Multithreaded caching Web proxy

Variants of Readers-Writers

- *First readers-writers problem* (favors readers)
  - No reader should be kept waiting unless a writer has already been granted permission to use the object
  - A reader that arrives after a waiting writer gets priority over the writer
- *Second readers-writers problem* (favors writers)
  - Once a writer is ready to write, it performs its write as soon as possible
  - A reader that arrives after a writer must wait, even if the writer is also waiting
- *Starvation* (where a thread waits indefinitely) is possible in both cases

Solution to First Readers-Writers Problem

- *First readers-writers problem* (favors readers)
  - No reader should be kept waiting unless a writer has already been granted permission to use the object
  - A reader that arrives after a waiting writer gets priority over the writer



Solution to First Readers-Writers Problem

- Figure shows a solution to the first readers-writers problem.
- Like the solutions to many synchronization problems, it is subtle and deceptively simple.
- The w semaphore controls access to the critical sections that access the shared object.
- The mutex semaphore protects access to the shared readcnt variable, which counts the number of readers currently in the critical section.
- A writer locks the w mutex each time it enters the critical section, and unlocks it each time it leaves.
- This guarantees that there is atmost one writer in the critical section at any point in time.
- On the other hand, only the first reader to enter the critical section locks w, and only the last reader to leave the critical section unlocks it.
- The w mutex is ignored by readers who enter and leave while other readers are present.
- This means that as long as a single reader holds the w mutex, an unbounded number of readers can enter the critical section unimpeded.

Readers:

```
int readcnt;    /* Initially = 0 */
sem_t mutex, w; /* Initially = 1 */

void reader(void)
{
   while (1)
   {
            P(&mutex);
            readcnt++;
            if (readcnt == 1) /* First in */
            P(&w);
            V(&mutex);

            /* Critical section */
            /* Reading happens  */

            P(&mutex);
            readcnt--;
            if (readcnt == 0) /* Last out */
             V(&w);
            V(&mutex);
   }
}
```

Writers:

rw1.c

```
void writer(void)
{
   while (1)
  {
      P(&w);

     /* Critical section */
     /* Writing happens  */

      V(&w);
   }
}
```

Crucial concept: Thread Safety

- Functions called from a thread must be *thread-safe*

- *Def:* A function is *thread-safe* iff it will always produce correct results when called repeatedly from multiple concurrent threads
- Classes of thread-unsafe functions:
  - Class 1: Functions that do not protect shared variables
  - Class 2: Functions that keep state across multiple invocations
  - Class 3: Functions that return a pointer to a static variable
  - Class 4: Functions that call thread-unsafe functions ☺

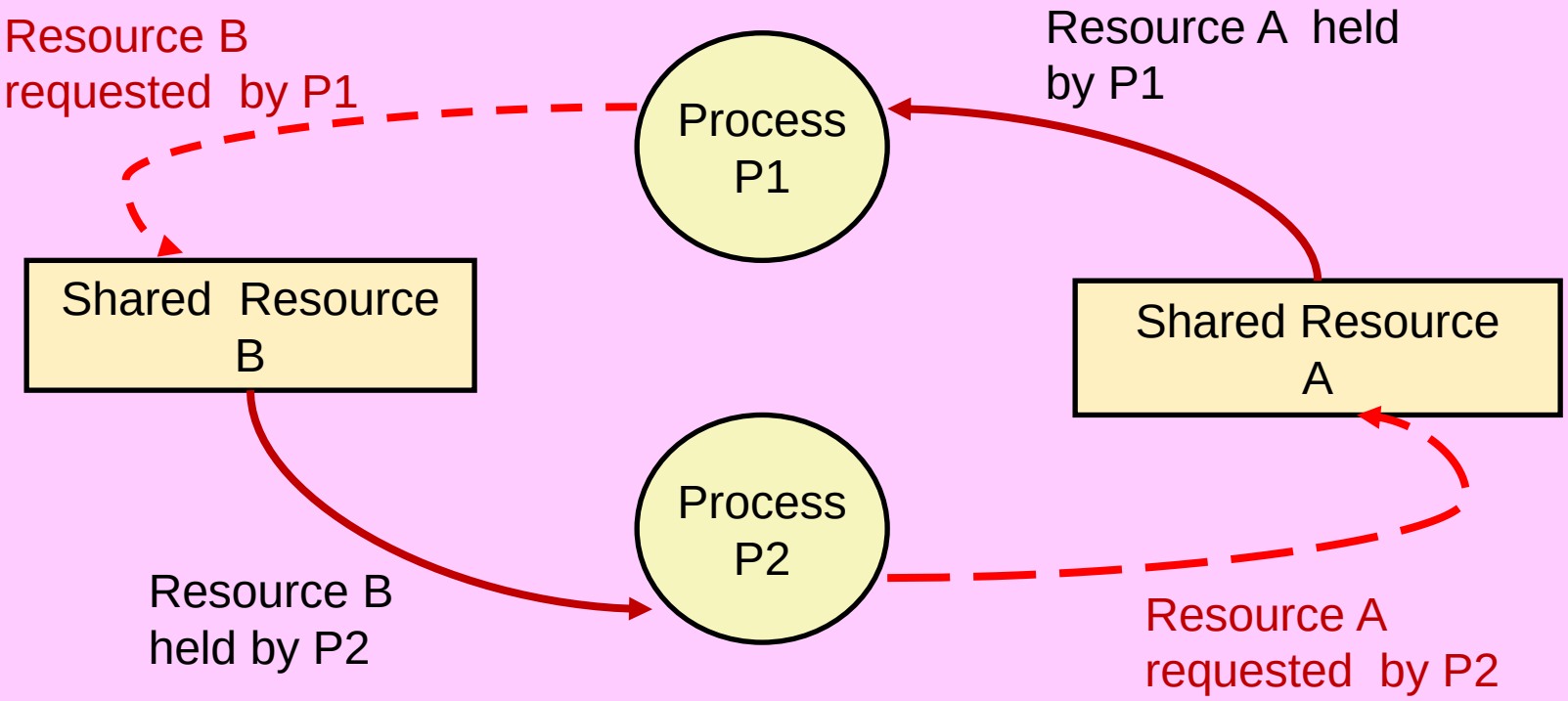
Another worry: Deadlock

- Def: A process is *deadlocked* iff it is waiting for a condition that will never be true

- Typical Scenario
  - Processes 1 and 2 need two resources (A and B) to proceed
  - Process 1 acquires A, waits for B
  - Process 2 acquires B, waits for A
  - Both will wait forever!

DEADLOCKS

Deadlock: improper resource allocation prevents forward progress



DEADLOCKS

- Semaphores introduce the potential for a nasty kind of run-time error, called deadlock, where a collection of threads are blocked, waiting for a condition that will never be true.
- The progress graph is an invaluable tool for understanding deadlock.
- For example, Figure shows the progress graph for a pair of threads that use two semaphores for mutual exclusion.
- From this graph, we can glean some important insights about deadlock:
  - The programmer has incorrectly ordered the P and V operations such that the forbidden regions for the two semaphores overlap.
- If some execution trajectory happens to reach the deadlock state d, then no further progress is possible because the overlapping forbidden regions block progress in every legal direction.
- In other words, the program is deadlocked because each thread is waiting for the other to do a V operation that will never occur.
  - The overlapping forbidden regions induce a set of states called the deadlock region.
- If a trajectory happens to touch a state in the deadlock region, then deadlock is inevitable.
- Trajectories can enter deadlock regions, but they can never leave. .

DEADLOCKS

- Deadlock is an especially difficult issue because it is not always predictable. Some lucky execution trajectories will skirt the deadlock region, while others will be trapped by it.
- Figure shows an example of each. The implications for a programmer are scary.
- You might run the same program 1000 times without any problem, but then the next time it deadlocks.
- Or the program might work fine on one machine but deadlock on another.
- Worst of all, the error is often not repeatable because different executions have different trajectories.

Deadlocking With Semaphores

```
int main()
{
   pthread_t tid[2];
   sem_init(&mutex[0], 0, 1);  /* mutex[0] = 1 */
   sem_init(&mutex[1], 0, 1);  /* mutex[1] = 1 */
   pthread_create(&tid[0], NULL, count, (void*) 0);
   pthread_create(&tid[1], NULL, count, (void*) 1);
   pthread_join(tid[0], NULL);
   pthread_join(tid[1], NULL);
   printf("cnt=%d\n", cnt);
   exit(0);
}
```

```
void *count(void *vargp)
{
   int i;
   int id = (int) vargp;
   for (i = 0; i < NITERS; i++) {
            P(&mutex[id]);
         P(&mutex[1-id]);
            cnt++;
            V(&mutex[id]);
         V(&mutex[1-id]);
   }
   return NULL;
}
```

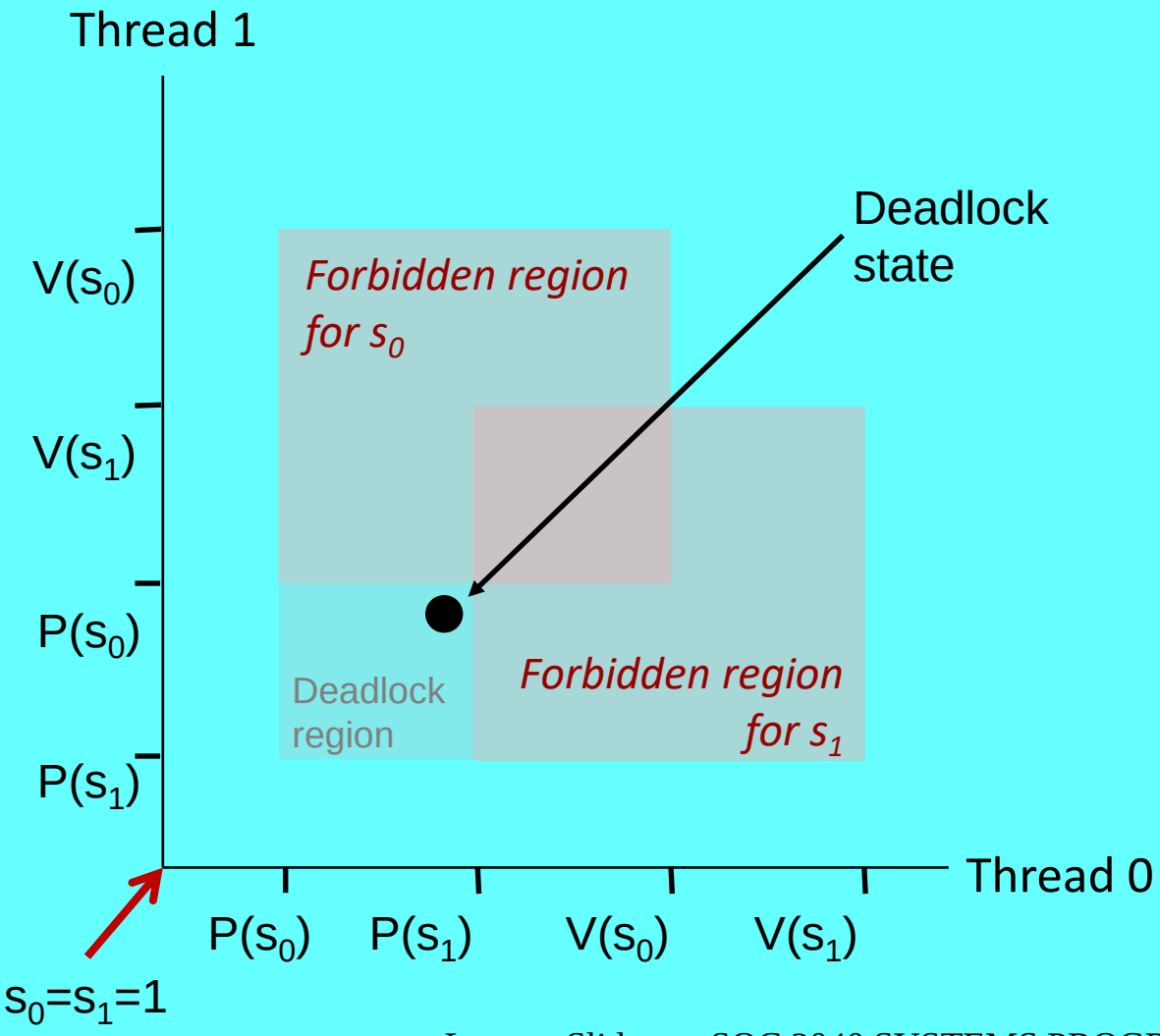
s0 ← mutex[0]
s1 ← mutex[1]

DEADLOCK

Tid[0]:
$P(s_0)$;
$P(s_1)$;
cnt++;
$V(s_0)$;
$V(s_1)$;

Tid[1]:
$P(s_1)$;
$P(s_0)$;
cnt++;
$V(s_1)$;
$V(s_0)$;

Deadlock Visualized in Progress Graph



Locking introduces the potential for *deadlock:* waiting for a condition that will never be true

Any trajectory that enters the *deadlock region* will eventually reach the *deadlock state*, waiting for either s_0 or s_1 to become nonzero

Other trajectories luck out and skirt the deadlock region

Unfortunate fact: deadlock is often nondeterministic (race)

DEADLOCKS

- Programs deadlock for many reasons and avoiding them is a difficult problem in general.
- However, when binary semaphores are used for mutual exclusion, then you can apply the following simple and effective rule to avoid deadlocks:
- Mutex lock ordering rule:
  - A program is deadlock-free if, for each pair of mutexes (s, t) in the program, each thread that holds both s and t simultaneously locks them in the same order.
  - For example, we can fix the deadlock in previous slide figure by locking s first, then t in each thread. Figure on the next slide shows the resulting progress graph.

Avoiding Deadlock

Acquire shared resources in same order

```
int main()
{
    pthread_t tid[2];
    Sem_init(&mutex[0], 0, 1);  /* mutex[0] = 1 */
    Sem_init(&mutex[1], 0, 1);  /* mutex[1] = 1 */
    Pthread_create(&tid[0], NULL, count, (void*) 0);
    Pthread_create(&tid[1], NULL, count, (void*) 1);
    Pthread_join(tid[0], NULL);
    Pthread_join(tid[1], NULL);
    printf("cnt=%d\n", cnt);
    exit(0);
}
```

```
void *count(void *vargp)
{
    int i;
    int id = (int) vargp;
    for (i = 0; i < NITERS; i++) {
        P(&mutex[0]); P(&mutex[1]);
        cnt++;
        V(&mutex[id]); V(&mutex[1-id]);
    }
    return NULL;
}
```

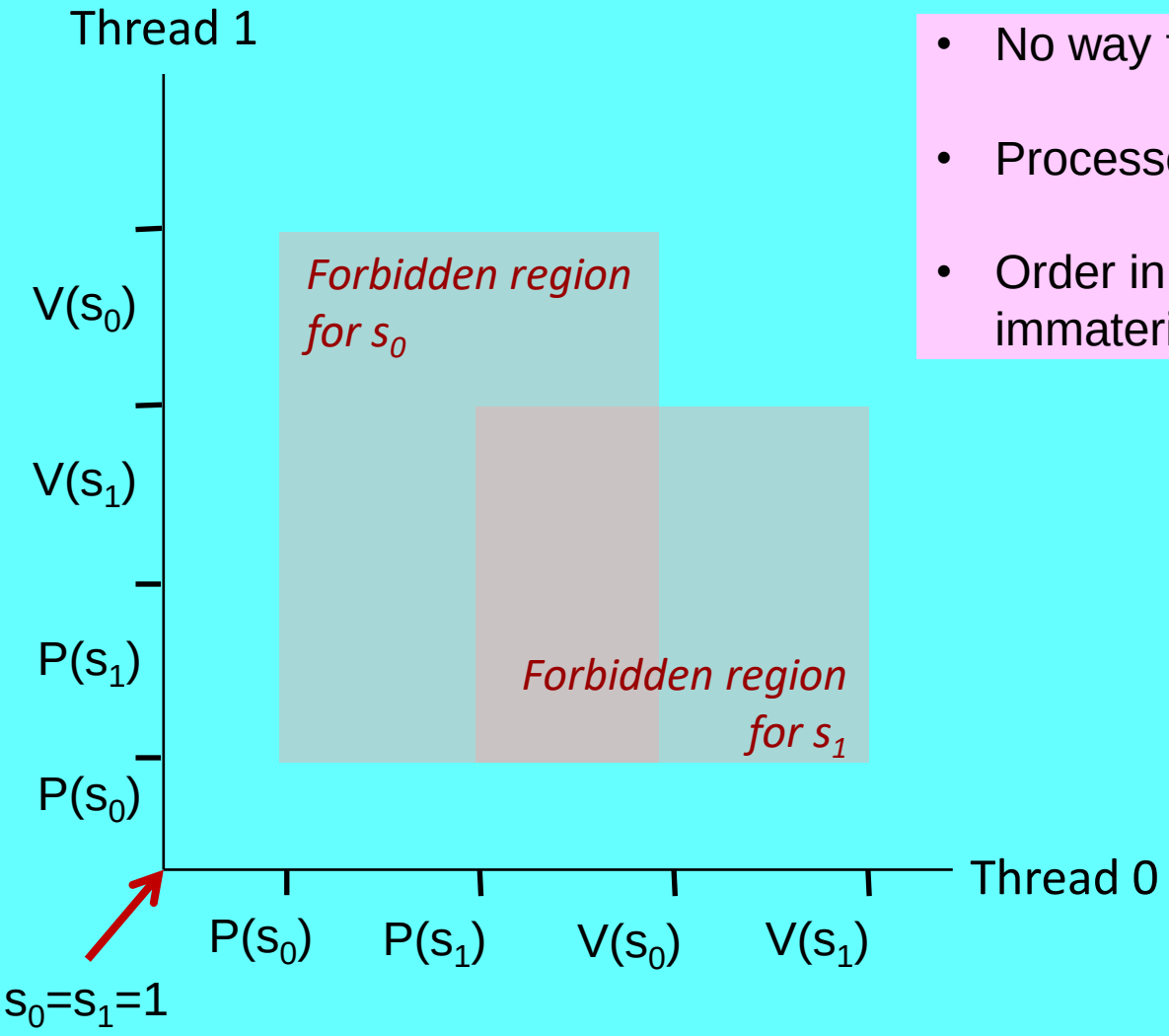
s0 ← mutex[0]
s1 ← mutex[1]

NO DEADLOCK

Tid[0]:
P(s0);
P(s1);
cnt++;
V(s0);
V(s1);

Tid[1]:
P(s0);
P(s1);
cnt++;
V(s1);
V(s0);

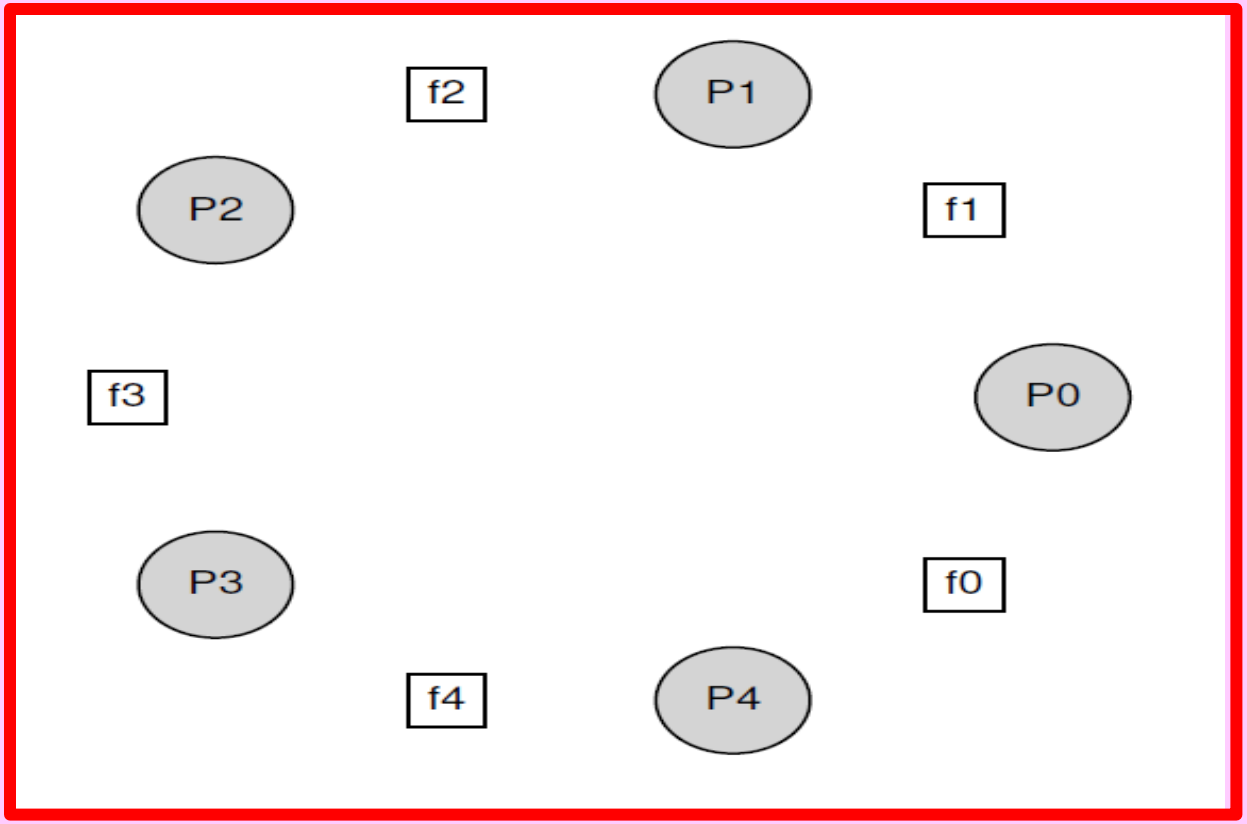
Avoided Deadlock in Progress Graph



- No way for trajectory to get stuck
- Processes acquire locks in same order
- Order in which locks released immaterial

Dining Philosopher's Problem

- One of the most famous concurrency problems posed, and solved, by Dijkstra, is known as the dining philosopher's problem .
- The problem is famous because it is fun and somewhat intellectually interesting; however, its practical utility is low.
- However, its fame forces its inclusion here; indeed, you might be asked about it on some interview.
- The basic setup for the problem is as shown in figure




```
while (1) {

think();

get_forks(p);

eat();

put_forks(p);

}
```

Dining Philosophers Problem

- Assume there are five “philosophers” sitting around a table.
- Between each pair of philosophers is a single fork (and thus, five total).
- The philosophers each have times where they think, and don’t need any forks, and times where they eat.
- In order to eat, a philosopher needs two forks, both the one on their left and the one on their right.
- The contention for these forks, and the synchronization problems that ensue, are what makes this a problem we study in concurrent programming.
- Here is the basic loop of each philosopher, assuming each has a unique thread identifier p from 0 to 4 (inclusive):

```
while (1) {
        think();
        get_forks(p);
        eat();
        put_forks(p); }
```

- The key challenge, then, is to write the routines get forks() and put forks() such that there is no deadlock, no philosopher starves and never gets to eat, and concurrency is high (i.e., as many philosophers can eat at the same time as possible).

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 12 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - **Randal E. Bryant and David R. O'Hallaron**
    3rd Edition, Global Edition Prentice Hall 2016
    ISBN 10:1-292-10176-8

SOC 2040
SYSTEM PROGRAMMING

END OF

WEEK 14 Lecture 1

CONCURRENT PROGRAMMING

WISH YOU ALL THE BEST

14 May 2024

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG